



**Università
di Genova**

DIBRIS DIPARTIMENTO
DI INFORMATICA, BIOINGEGNERIA,
ROBOTICA E INGEGNERIA DEI SISTEMI

From natural language to Prolog

Laura Fuciarelli

Master Thesis

Università di Genova, DIBRIS Via Opera Pia, 13 16145 Genova, Italy
<https://www.dibris.unige.it/>



MSc Computer Science
Data Science and Engineering Curriculum

From natural language to Prolog

Laura Fuciarelli

Advisor: Prof. Viviana Mascardi

Examiner: Prof. Giorgio Delzanno

March, 2026

*I dedicate this thesis to my mom, who was always putting my needs before her own,
especially on the tough days. I will never be able to thank her enough.*

Abstract

Currently, large language models (LLMs) and generative artificial intelligence (GenAI) are among the principal approaches used for a range of tasks, including translating natural language into a formal representation. In this context, we introduce Talk2Prolog, a faster, lighter, and more transparent alternative for translating an English sentence into a Prolog fact, rule, or query. Talk2Prolog can run locally on a standard laptop, guarantees interoperability and consistency due to its coherent translation style, and aligns with trustworthy, sustainable, and explainable artificial intelligence principles. Talk2Prolog was built based on a methodology that formalizes common Prolog programming practices, which we derived from a dataset we created using a bottom-up approach. This dataset contains pairs of Prolog expressions and their English meanings obtained from tutorials, university lessons, and GPT. We also included sentences and the corresponding translations obtained with Talk2Prolog in the dataset, some of which were reviewed by Prolog experts and Prolog newcomers. We tested Talk2Prolog against GPT-5.2 using the dataset, evaluating syntax, semantics, and time performance for the translation process. Despite its lightweight implementation, Talk2Prolog demonstrates promising results, with an accuracy of 88.3% in experimental semantic translation tasks.

Table of Contents

Chapter 1	Introduction and Motivation	13
Chapter 2	Background	17
2.1	Prolog basics	17
2.2	Linguistics basics	18
2.2.1	Sentences	18
2.2.2	Clauses	19
2.2.3	Verbs	21
2.3	Natural Language Processing (NLP)	23
Chapter 3	Related Work	26
3.1	Research questions	26
3.2	Sources	27
3.3	Technology synopsis	27
3.3.1	Controlled natural language	28
3.3.2	Symbolic technologies	29
3.3.3	Neuro symbolic technologies	31
3.3.4	Neural technologies	32
3.3.5	Datasets and benchmarks	33
3.4	Analysis	34

Chapter 4	Dataset Creation	36
4.1	Sources for the dataset	38
4.2	Selected examples	39
4.2.1	Facts Websites	39
4.2.2	Facts Mix	39
4.2.3	Facts Variables	39
4.2.4	Queries Websites	40
4.2.5	Queries Mix	40
4.2.6	Rules Websites	41
4.2.7	Rules Mix	41
4.2.8	0-arity facts (GPT)	41
4.2.9	1-arity facts (GPT)	42
4.2.10	2-arity facts (GPT)	42
4.2.11	Running examples	42
4.2.12	Semantic examples	46
Chapter 5	Practices-driven Methodology and Design	48
5.1	Simple sentences	50
5.1.1	Expletive constructions	50
5.1.2	SV clauses	51
5.1.3	SVA clauses	51
5.1.4	SVO clauses	52
5.1.5	SVC clauses	53
5.1.6	Simple negative sentences	53
5.1.7	Simple passive sentences	54
5.1.8	Simple interrogative sentences	54
5.2	Complex sentences	58
5.2.1	Independent clause	58

5.2.2	Dependent clause	58
5.3	Shared features	60
5.3.1	Anaphora	60
5.3.2	Variables	61
5.3.3	Nesting	62
5.3.4	Subject-verb agreement	63
5.3.5	Prepositions	63
5.3.6	Nouns	63
5.3.7	Quantities	64
5.4	Summary	65
5.5	Design of Talk2Prolog	66
Chapter 6	Implementation	68
6.1	Common parameters	69
6.2	Pre-processing	71
6.2.1	Coreference Resolution	71
6.2.2	Sentence to clauses	72
6.3	Semantic parsing	73
6.4	Prolog handling	73
6.4.1	Query	74
6.4.2	Insert	74
Chapter 7	Experiments	76
7.1	Talk2Prolog settings	76
7.2	GPT settings	77
7.3	Syntactic testing	79
7.3.1	Well-formed expressions	79
7.3.2	Logical equivalence	80

7.3.3	String similitary	82
7.4	Semantic testing	85
7.4.1	Yes/no questions	86
7.4.2	Question words	87
7.4.3	Passive vs active voice	88
7.4.4	Modus Ponens	89
7.4.5	Modus Tollens	89
7.4.6	Hypothetical Syllogism	90
7.4.7	Conjunction	91
7.4.8	Disjunction	93
7.5	Performance	95
7.6	Results spreadsheet structure	96
7.7	Comparison between Talk2Prolog and GPT 5.2	100
Chapter 8	Conclusions and Future Work	102
Chapter 9	Appendix: Selected Code Fragments	107
	Bibliography	111

Chapter 1

Introduction and Motivation

In November 2022, ChatGPT was launched to the general public. Its impressive and unprecedented capabilities dramatically changed the public perception of Artificial Intelligence (AI), to the point that people who are not Computer Science professionals often seem to associate the term “AI” to generative AI (GenAI) and large language models (LLMs).

In particular, the term “AI” is often associated with natural language processing (NLP), which is not surprising, since language is recognized by many scientists and philosophers as one of the features that make humans unique [Den97].

Currently, in fact, as well as in the early days of AI, the possibility to instruct machines using natural language is more and more fashionable. In fact, this is one of the problems that Agentic AI and last-generation LLMs promise to solve: moving from general natural language to an unambiguous, artificial programming language, which could potentially kill white-collar jobs as we know them ¹.

Regardless of what people intend by the term “AI” and of its clear potential, surveys on public opinion on “AI” show that people do not trust it for several reasons: it is not sustainable from an environmental, financial and geopolitical point of view, and creates significant privacy issues, especially considering the black box nature of deep learning based technologies ².

In this current AI landscape, this thesis aims to present Talk2Prolog, a software that emerges as an alternative to LLMs and Gen AI, such as GPT, considering the task of

¹<https://shumer.dev/something-big-is-happening>, accessed on March 10, 2026.

²<https://hai.stanford.edu/ai-index/2025-ai-index-report/public-opinion>, <https://www.turing.ac.uk/news/publications/understanding-public-attitudes-ai-second-survey>, <https://www.pewresearch.org/global/2025/10/15/how-people-around-the-world-view-ai/>, accessed on March 10, 2026.

moving from a controlled subset of natural language to an artificial programming language.

Talk2Prolog was designed using a well-founded rule-based approach based on a methodology, supported by an inspectable and debuggable implementation, which makes it explainable and copes with the lack of trust.

The main characteristics of Talk2Prolog are being lightweight, frugal, and privacy-preserving as possible. By frugal, we mean that the design and implementation of the software requires and consumes only the resources that are truly needed, no more. It is both more sustainable on a macro-level for the planet, but also on a micro-level, since it is a software that can be easily installed and run locally on a modern personal computer.

Therefore, it intentionally does not use large language models (LLMs) and transformers, avoiding complex machine and deep learning techniques as much as possible.

From a practical point of view, Talk2Prolog consists of a software written in Python that expects as input a knowledge base and a sentence written in a simplified version of English. It uses a preprocessor to normalize the input sentence, a semantic parser to individuate the terms, and a Prolog interpreter to communicate with the knowledge base.

Despite its frugality and some limitations in terms of input flexibility, Talk2Prolog demonstrated to be a fast and consistent tool, especially when compared to LLMs, such as GPT 5.2.

The domains in which Talk2Prolog can be used are the following:

- Belief-Desire-Intentions (BDI) multi-agent systems [RG95], where Talk2Prolog can be used to translate natural language sentences into intelligent agents' beliefs, is the main domain we considered while designing the software. In our experience, BDI systems often do not maintain time-stamped beliefs, since they are driven by a continuous reasoning cycle. This means that beliefs available in the belief base at time t might be different from those at time $t + 1$, because of belief updates. Also, BDI agents often do not keep beliefs on the other agents' beliefs. For these reasons, these beliefs representations do not need temporal and modal logic.
- Talk2Prolog could be used in situations where interoperability between systems is fundamental. In fact, Talk2Prolog follows a deterministic translation methodology driven by common Prolog programming practices, which means that there is a coherent syntactic way to represent in Prolog the same type of linguistic features. It allows to keep consistency in the translation if the system is used by different users and in different moments.
- Talk2Prolog could also be used for systems in which having a lightweight software to perform translations locally in not particularly powerful computers is necessary.

Talk2Prolog intentionally does not use large language models (LLMs) and transformers in general, avoiding complex machine and deep learning techniques as much as possible, which makes it a lighter and faster to run when compared to LLM.

- Systems that need to be explainable, for privacy, safety, critical systems, or other reasons, are also a domain in which Talk2Prolog could be used. The consistency of the translations also guarantees determinism, preservation of meaning, and behaviour coherence.

The code and datasets can be found on my GitHub ³.

This thesis is divided into the following chapters:

- Chapter 2: Background, in which we will explain the fundamental concepts for understanding how Talk2Prolog works. This chapter will include the basics of Prolog, English grammar, and natural language processing (NLP). For the last point, we will focus on Python, a programming language widely used for NLP tasks and chosen for the implementation of Talk2Prolog.
- Chapter 3: Related work, in which we will introduce the currently available technologies and techniques for translating from a natural to a formal language, some of which inspired our own work.
- Chapter 4: Dataset Creation, in which we will explain how we created a custom dataset with Prolog representations and respective English meaning for creating a methodology, test our software, and compare the results and performance of Talk2Prolog with other tools. The sources for this dataset are tutorials, university lessons and GPT, which is by definition the synthesis of the Prolog content available online. We will also discuss the feedback provided by Prolog experts and Prolog newcomers on examples for the dataset created using Talk2Prolog.
- Chapter 5: Practices-driven Methodology and Design, in which we will explain a methodology created to formalize how to translate a sentence in English into a plausible Prolog representation based on common practices adopted by Prolog users. These practices were retrieved from the dataset with a bottom-up approach.
- Chapter 6: Implementation, in which we will explain in details how we used Python and, in particular, the spaCy library to implement the tool Talk2Prolog, which is based on the methodology we created, concentrating on different parts of the software.

³GitHub, accessed on March 10, 2026.

- Chapter 7: Experiments, in which we will explain the configuration of the tools we used, Talk2Prolog and GPT-5.2, and the syntactic, semantic, and time performance testings we conducted, comparing the results we achieved with each tool.
- Chapter 8: Conclusions and Future Work, in which we will summarize the outcomes of the thesis work and the possible improvements that emerged during the development of the dataset, methodology and software.
- Chapter 9: Appendix: Selected Code Fragments, in which we will show the most relevant code fragments used for the implementation and testing.

Chapter 2

Background

2.1 Prolog basics

Prolog is a declarative and logic programming language.

The *declarative programming paradigm* expresses the logic of a computation without describing its control flow.

The *logic programming paradigm* relies on formal logic to describe computations. In Prolog, first-order logic is used.

A Prolog program consists of a set of *facts* and *rules*, which constitute a knowledge base (also known as context). They are used to define relations and to infer new information.

A computation is initiated by posing a query to the program, which Prolog attempts to satisfy using its knowledge base.

Facts and rules are both *Horn clauses*.

A Horn clause consists of the head, on the left-hand side of the logical implication (represented in Prolog with the symbol “:-”), and the body, on the right-hand side.

`head :- body`

The head is the conclusion and has at most one positive predicate. This means it cannot be negative or contain logical connectives.

The body instead contains the premises and can have multiple positive or negated predicates. These predicates are linked by logical connectives.

A fact in Prolog is a bodiless Horn clause, while a rule contains both the head and the body.

A Prolog *term* is the fundamental building block of all data and expressions in Prolog.

A term can be:

- Atom: a constant symbol (e.g. apple or ‘New York’).
- Number: an integer or floating-point number (e.g. 42, 3.14).
- Variable: a placeholder for unknown value and can match/be bound to any Prolog term (e.g. X or the anonymous variable `_`, which matches anything but does not bind).
- Compound term: a functor with arguments (e.g. book(title, author, year)).
- List: an ordered collection of elements(e.g. [a, b, c]).

Functor is the name of a compound term or a predicate. A *predicate* is the a relation between terms.

Negation as failure in Prolog means that a statement is considered false if Prolog fails to prove it true.

Prolog operates under the *closed-world assumption*. Therefore, it treats missing information as negation without explicitly storing negative facts.

Negation as failure is closely related to this assumption: anything that cannot be proven true (i.e., not present in the knowledge base) is assumed to be false.

2.2 Linguistics basics

In this section, we will cover the basics of English grammar.

2.2.1 Sentences

A *sentence* is a string of words that expresses a complete thought in natural language.

It must contain both a subject and a verb.

There are four main types of sentences:

1. Declarative: This type of sentence is used to state a fact. For example, “Alice is happy.” is a declarative sentence.

2. Interrogative: This type of sentence is used to ask a question. For example, “Is Alice happy?” is an interrogative sentence.
3. Exclamatory: This type of sentence is used to add emphasis or show emotion, urgency, or high volume. It is a modified version of a declarative sentence. For example, “Alice is happy!” is an exclamatory sentence.
4. Imperative: This type of sentence is used to tell a command to someone or something. The subject in this case is assumed and does not need to be included explicitly in the sentence. For example, “Be happy.” is an imperative sentence.

There are two grammatical voices for sentences:

- The active voice, which is a grammatical structure where the subject of the sentence covers the semantic role of the agent, so it performs the action of the verb.
- The passive voice, which is a grammatical structure where the subject of the sentence covers the semantic role of the patient, so it receives the action of the verb.

The passive constructions consists of a stative verb (a verb that describes a state of being and is usually the verb “to be”, but sometimes the verb “to get” can also be used) and the past participle of the verb denoting the action.

The appearance of the agent inside the sentence is optional, but can be specified by using a prepositional phrase with the preposition “by”. For the methodology, we considered the cases in which the agent is present.

2.2.2 Clauses

Sentences are formed by *clauses*.

A clause is a group of words that contains a subject and a verb. The verb forms a relationship that expresses an action or state of being of the subject.

Clauses can function as an independent sentence, but it is not always the case. In fact, sentences might contain multiple clauses, some of which depend on another clause to express a complete thought.

For this reason, sentences can be further classified based on their structure:

1. Simple sentences are composed by a single independent clause. For example, “Alice is happy.” is a simple sentence.

2. Complex sentences combine an independent clause with one or more dependent (also known as subordinate) clauses. In this type of sentence, subordinating conjunctions (e.g. “if”, “when”, “after”, etc.) are used to connect a dependent clause to an independent one. For example, “Alice is happy if it is sunny.” is a complex sentence.
3. Compound sentences combine two independent clauses using a coordinating conjunction (e.g., the FANBOYS, “for”, “and”, “nor”, “but”, “or”, “yet”, “so”) or using a semicolon. For example, “Alice is happy and the sky is blue.” is a compound sentence.
4. Compound-complex sentences have at least two independent clauses and at least one subordinate clause. For example, “Alice is happy and the sky is blue if it is sunny.” is a compound-complex sentence.

Clauses can also be classified in different categories.

For this thesis, I used as a reference the grammar explained in “A Comprehensive Grammar of the English Language” [Hud88], which recognizes seven types of clause structure, based on the elements present in the clauses.

The clause elements identified by this grammar are:

1. Subject (S)
2. Verb (V)
3. Object (O)
 - (a) Direct
 - (b) Indirect
4. Complement (C)
 - (a) Subject
 - (b) Object
5. Adverbial (A)

The clause structures identified by the grammar are:

1. SV: subject + (intransitive) verb
2. SVO: subject + (monotransitive) verb + (direct) object

3. SVC: subject + (copular) verb + complement
4. SVA: subject + (copular) verb + adverbial
5. SVOO: subject + (ditransitive) verb + (indirect) object + (direct) object
6. SVOC: subject + (complex transitive) verb + (direct) object + complement
7. SVOA: subject + (complex transitive) verb + (direct) object + adverbial

2.2.3 Verbs

Verbs can also be classified accordingly to different types.

The two most general categorization classifies verbs as *lexical* or *auxiliary*.

Lexical verbs, also known as main verbs, express a clear meaning and do not rely on any other verb to be grammatical. For example, if we consider the sentence “Alice reads a book.”, “reads” is a lexical verb.

Auxiliary verbs, also known as helping verbs, do not contain much meaning and are instead used to show grammatical features (e.g., tense, aspect, or modality). They are usually placed directly before a lexical verb.

For example, if we consider the sentence “Alice has read a book.”, “has” is an auxiliary verb (and “read” is a lexical verb).

Auxiliary verbs can be recognized due to two unique features:

- They require subject-verb inversion when forming a question (e.g. “Has Alice read a book?”).
- They are followed by the negator “not” in negative constructions (e.g. “Alice has not read a book.”).

There are also more specific categories for classifying verbs. For the scope of this thesis, we will focus on transitivity and the copula.

2.2.3.1 Transitivity

A transitive verb is a verb that requires an object to complete its meaning. For example, “to write” is a transitive verb (“Bob writes a letter.”).

An intransitive verb, instead, is the exact opposite: it is a verb that does not complete its meaning if an object is present. For example, “to laugh” is an intransitive verb (“Alice and Charlie laughed.”).

In English, some verbs can act as transitive or intransitive depending on the context of the sentence. For example, “to eat” can act both as a transitive and as intransitive verb based on the context (“Bob eats salad.” and “Alice eats early.”, respectively).

There are some subcategories for transitivity:

- A monotransitive verb takes only a direct object (e.g. the verb “to read” in “Charlie reads a new book.”)
- A ditransitive verb takes both a direct and an indirect object (e.g. the verb “to buy” in “Bob bought himself a new pen.”).
- A complex transitive takes a direct object and an object complement (e.g. the verb “to find” in “Alice found the story interesting.”).

2.2.3.2 Copula

Copular verbs, also known as linking verbs or copula, require a predicative complement that provides more information about the clause subject.

The predicative complement can be:

- Predicative nominal when a noun, noun phrase, or pronoun functions as a subject complement. For example, in the sentence “Alice is a student.”, “a student” is a predicative nominal.
- Predicative adjective when an adjective or an adjective phrase functions as a subject complement. For example, in the sentence “Alice is a happy.”, “happy” is a predicative adjective.

In both cases, the copula is the verb “is”.

Copular verbs are intransitive, so they do not take an object. They also do not contain much meaning, being more of a grammatical nature by connecting the subject with the complement.

2.3 Natural Language Processing (NLP)

Natural language processing (NLP) is a subfield of computer science and artificial intelligence (AI) focused on tools and methodologies that enable computers to understand, process, and communicate with human language.

Some examples of NLP uses are chatbots, language translation systems, and voice recognition softwares.

Currently, NLP is often associated with language models (LMs), which are computational models that predict sequences in natural language. More specifically, large language models (LLMs), that are predominantly based on transformers trained on massive vast amount of data (frequently datasets that consists of texts scraped from the public internet) and are particularly useful for generating text.

Still, there are many other NLP tasks that are not related to text generation, for example:

- Part-of-speech tagging: Assigning grammatical categories (e.g. noun, verb, etc.) to each word in a sentence.
- Sentence dependency parsing: Analyzing grammatical relationships between words in a sentence (e.g. noun subject, direct object, etc.).
- Lemmatization: Reducing words to their base (also known as dictionary) form (e.g. “running” to “run”).
- Named entity recognition: Identifying and classifying proper names into categories (e.g. people, place, or organization).

The programming language Python is often preferred for dealing with NLP tasks for several reasons, some of them are:

- Python’s straightforward semantics and syntax make the learning curve gentler for non-programmers, and the projects easier and faster to develop.
- Python is a solid programming language, and provides support for integration with tools developed in other programming languages.
- Python offers a huge variety of NLP tools and libraries, such as NLTK and spaCy, which are both free and open-source.

NLTK is the pioneer NLP library for Python. It was initially released in 2001 and is still widely used in teaching, research, and rule-based NLP tasks.

NLTK is powerful due to its modularity, extensibility, and accessibility. However, it is not considered very efficient, as it prioritizes breadth of coverage over optimization and is implemented in pure Python.

For this reason, in 2015, the library spaCy [Vas20] was released as a production-ready alternative to NLTK, which uses statistics and word embeddings to provide useful information for NLP tasks. It is optimized for speed and accuracy by using Cython instead of pure Python.

Opposed to NLTK, which has an education focus, spaCy was primarily designed for industrial-scale applications. In fact, it can efficiently process large volumes of text without losing reliability, modularity, and accessibility.

SpaCy is also a less manual tool compared to NLTK. On the one hand, it limits the developer’s possibilities, but on the other hand, it is faster to develop since it usually requires fewer lines of code to achieve the same result.

After considering the pros and cons of both libraries with reference to our project, spaCy was chosen as the main NLP tool for the development of Talk2Prolog for the following reasons:

- Efficiency and accuracy were the two most fundamental requirements of our tool, and spaCy excels in these aspects.
- We had limited time to develop Talk2Prolog, so we needed to choose a tool that had a smooth learning curve.
- We only needed to extract common linguistic features, such as part-of-speech tagging, sentence dependency parsing, lemmatization, and named entity recognition. All of these are already supported by spaCy through its pre-trained models.
- Tools not available directly through spaCy could be added as plugins and extensions thanks to Spacy Universe, such as “pyinflect” for word inflections and “numerizer” for dealing with numbers expressed in written form.
- Although spaCy uses neural-network–based statistical models, it relies on a shared task-specific convolutional neural network (CNN) architecture to perform several tasks simultaneously, including syntactic dependency parsing, part-of-speech tagging, and named entity recognition. Hence it represents a relatively simpler deep-learning architecture when compared to transformers used in generative AI, for example.

We discarded other libraries (e.g., HuggingFace, Stanza, etc.) mainly for the following reasons:

- Smaller community and less material available.
- Greater use of deep-learning techniques, especially transformers, that we wanted to avoid as much as possible.

Chapter 3

Related Work

3.1 Research questions

Our main research question reflects our overall goal. We aim to understand:

(G) How to formalize Prolog programming practices in order to develop an implementation capable of translating natural language sentences into Prolog expressions?

In order to review the state of the art based on this question, it is necessary to re-articulate the research question (G) in terms of more detailed secondary research questions:

- (Q1) What methodologies and tools are actually available for translating natural language text to a formal language of a logical system, and how do they compare in terms of accuracy and efficiency?
- (Q2) How do logic translation methodologies and tools handle natural language ambiguity?
- (Q3) What datasets and benchmarks, if any, are commonly used to evaluate logic translation tools?

The above questions drive the process of research, by articulating the way in which papers and technologies are looked for, examined, and possibly selected for the review.

3.2 Sources

We analyzed in details 15 sources, mostly papers written by Prolog researchers, and some complementary papers as well.

We decided to discard papers that are not directly involved in the process of translation from natural language to a formal representation based on logic. In particular, we discarded articles focused on integrating components to enhance language models logical reasoning, but did not feature logic translation as its core objective.

We searched for relevant papers on Google Scholar and we also took into account papers suggested by experts via an online survey (see Section 4.2.11).

Regarding Google Scholar, most of the articles were found using the following combinations of the keywords:

- (KW1) nlp AND prolog generation
- (KW2) generation AND prolog
- (KW3) llm AND prolog
- (KW4) llm AND first-order logic
- (KW5) controlled natural language

3.3 Technology synopsis

Technologies for logic translation of natural language text can be classified according to three categories:

- Symbolic: approaches based on symbolic methods, such as knowledge representation, logical reasoning, and problem-solving algorithms.
- Neural: approaches based on neural networks and deep learning,
- Neuro-symbolic: hybrid approaches which use both symbolic and neural components.

There is often no sharp boundary between symbolic, neural, and neuro-symbolic approaches, as these paradigms frequently overlap and blend in practice, but we decided to use this classification as an overall guideline.

Technologies belonging to each sub-category are presented in chronological order and accompanied by a short description and URL referencing the projects home pages, aiming at providing an answer to one or more of the research questions listed in Sec. 3.1.

In addition to specific software and methodologies, we also analyzed datasets and benchmarks containing natural language text paired with formal logic representation. Stand-alone datasets are present in a section a part.

We also included a section to discuss controlled natural language, in particular, Logical English and Attempto.

3.3.1 Controlled natural language

Controlled natural language (CNL) [Kuh14] is a subset of natural language that can be accurately and efficiently processed by a computer, but is expressive enough to allow natural usage by non-specialists.

Attempto Controlled English (ACE) [FKK08] is an example of CNL. In particular, it is a precisely defined subset of English that can automatically and unambiguously be translated into first-order logic. Thus, ACE is human- and machine-comprehensible.

Although ACE was originally intended to specify software, it has been used as a general knowledge representation language in several application domains, most recently for the semantic web.

ACE is supported by a number of tools, predominantly by the Attempto Parsing Engine (APE) that translates ACE texts into Discourse Representation Structures (DRS), a variant of first-order logic.

Another example of CNL is Logical English (LE) ¹, which is also a subset of English that is both human- and machine-comprehensible.

LE is inspired in part by the observation that well-written legal documents often have the form of logic programs. For example, the paper “Logical English for Legal Applications” [KDC21] shows how LE meta-predicates can be used to represent propositional attitudes, such as permission, obligation, notification of a message and designation of the occurrence of an event. In the paper “The British Nationality Act as a logic program” (1986) [SSK⁺86], LE was used to implement the British Nationality Act to test the suitability of Prolog logic for expressing and applying legislation.

While ACE is intended for general-purpose representation and reasoning, LE is syntactic sugar for programs written in a logic programming language, such as Prolog, Datalog, or

¹<https://www.doc.ic.ac.uk/~rak/papers/LPOP.pdf>

Answer Set Programming (ASP).

Datalog [CGT89] is a query language for deductive databases. It is syntactically a subset of Prolog, but since it uses a bottom-up rather than top-down evaluation mode, it yields different behavior and properties from Prolog.

ASP instead is a declarative programming oriented towards difficult (primarily NP-hard) search problems [Lif19].

3.3.2 Symbolic technologies

3.3.2.1 Definite Clause Grammars (DCG) (1995)

In “Specifying Logic Programs in Controlled Natural Language” [FS95], a controlled natural language is proposed to bridge the gap between being accurate and efficiency enough to specify programs that be processed by a computer, but at the same time expressive enough to allow a natural usage by non-specialists.

The specifications written in controlled natural language by the user are translated into Prolog clauses (hence becoming formal and executable) by using Definite Clause Grammar (DCG) [PW80] enhanced by feature structures.

The features structures are written using GULP (Graph Unification Logic Programming) [Cov94], an extension that adds to Prolog two operators and a number of system predicates. Inter-text references present in the specifications (e.g. anaphora) are resolved with the help of discourse representation theory (DRT).

The resulting Prolog clauses are added to a knowledge. The user can ask yes/no and wh-questions formulated using controlled natural language. These questions are translated into Prolog queries, and answered with the help of a knowledge base.

3.3.2.2 Sequent-calculus (2008)

The paper “A Higher Order Logic Parser for Natural Language Implemented in Lambda Prolog” [Kre88] describes an implementation for F.C.N. Pereira’s idea of using a sequent-calculus like system ² to produce Montague semantics [DWP12] from natural language.

The lefthand sequent can be interpreted as a set of constraints under which a given sentence fragment has a certain interpretation. Pereira presents a few complementary ”discharge-rules” to reduce the number of such constraints. These conditional interpretations constitute a uniform way to represent partial knowledge during the parsing process.

²<https://www.cs.cmu.edu/~fp/courses/atp/handouts/ch3-seqcalc.pdf>

Implementation is done using Lambda Prolog, a higher-order Prolog extension that supports the lambda-tree syntax approach to representing and manipulating formal syntactic objects [Qi09].

3.3.2.3 Concept2Text (2024)

The paper “Concept2Text: an explainable multilingual rewriting of concepts into natural language” [BPF⁺24] focuses on the opposite task of Talk2Prolog: translating abstract concepts into natural language, after an initial step of interpreting and abstracting features from raw information, such as data series.

Concept2Text is presented as an explainable and modular alternative to LLMs, which, despite their impressive natural language generation capabilities, have trustworthiness that can be difficult to verify.

The paper explores a Prolog/CLP-based rewriting system designed to interpret concepts expressed in terms of classes and relations derived from a generic ontology, generating text in natural language. The core idea revolves around processing a tree that describes a concept to derive an equivalent grammar tree containing a well-formed sentence. This process is driven by a uniform rule-based rewriting system. During rewriting, trees undergo structural and semantic modifications and different sets of rules govern distinct stages in the transformation of the input tree.

DCGs were initially considered for this task instead of the native tree-to-tree rewriting system, but two limitations were encountered:

1. DCGs naturally model the structure of a specific tree rather than the rewriting relationships of a subtree into a new subtree.
2. In cases involving multiple alternative outputs of a rule (variants), explicit control over the selection process is necessary, as opposed to allowing the SLD resolution to explore all possibilities.

3.3.2.4 CNLWizard (2025)

The paper “A General Framework for Representing Controlled Natural Language Sentences and Translation to KR Formalisms” [CDMT25] introduces a general framework that generates grammars for target representation languages, enabling the translation of problems stated in controlled natural language (CNL) into knowledge representation (KR) formalisms (such as ASP).

The related implementation, CNLWizard, processes inputs described in a simple YAML-based language and automatically generates the grammar for the CNL, along with a set of pre-implemented imperative functions that significantly reduce the time and effort needed to create custom grammars.

CNLWizard operates through two main computational steps.

- The first step corresponds to the framework and consists of generating target grammar and functions.
- The second step is responsible for the actual translation of a CNL file into a target language representation.

3.3.3 Neuro symbolic technologies

3.3.3.1 Combinatory Categorical Grammar (CCG) (2012)

The paper “Learning to Map Sentences to Logical Form: Structured Classification with Probabilistic Categorical Grammars” [ZC12] addresses the problem of mapping natural language sentences to lambda-calculus encodings of their meaning.

It describes a learning algorithm that takes as input a training set of sentences labeled with expressions in the lambda calculus. The algorithm induces a grammar that maps sentences to logical form, using as formalism combinatory categorical grammar (CCG). The CCG formalism has an elegant treatment of a wide range of linguistic phenomena, such as coordination and relative clauses.

CCGs are generalized to their probabilistic version with a log-linear model that represents a distribution over syntactic and semantic analyses conditioned on the input sentence.

3.3.3.2 NL2ASP (2024)

The paper “Towards Automatic Composition of ASP Programs from Natural Language Specifications” [SKR24] presents:

- NL2CNL dataset, designed to develop and assess tools for ASP automatic coding. It includes graph-related problem specifications in natural language and corresponding CNL propositions
- NL2ASP tool, a two-step architecture for generating ASP programs from natural language specifications.

NL2ASP first uses neural machine translation to transform natural language into Controlled Natural Language (CNL) statements. Then CNL statements are converted into ASP code using the CNL2ASP tool [CDM⁺24].

3.3.3.3 CLOVER (2025)

The paper “Divide and Translate: Compositional First-Order Logic Translation and Verification for Complex Logical Reasoning” [RKLY25] introduces CLOVER (Compositional First-Order Logic Translation and Verification), a neurosymbolic approach created to enhance complex logical reasoning in LLMs.

CLOVER consists of first using an LLM to parse a natural language sentence into newly defined logical dependency structures. These structures consist of an atomic subsentence and its dependents. Then, CLOVER sequentially translate the parsed subsentences to FOL formulas.

Finally, since multiple logical dependency structures and sequential translations are possible for a single sentence, two SAT-based FOL verification algorithms are also introduced to ensure more reliable results.

3.3.4 Neural technologies

3.3.4.1 LogicLLaMA and MALLS (2023)

The paper “Harnessing the Power of Large Language Models for Natural Language to First-Order Logic Translation” [YXP⁺23] introduces LogicLLaMA, a LLaMA-7B model fine-tuned for translating natural language sentences to first-order logic (NL-FOL translation).

The dataset MALLS (large language Model generAted NL-FOL pair) is also introduced in the paper. It is used to train LogicLLaMA and contains 34K high-quality and diverse sentence-level NL-FOL pairs collected from GPT-4.

3.3.4.2 LLASP (2024)

In “LLASP: Fine-tuning Large Language Models for Answer Set Programming” [CCM⁺24], a finetuned lightweight model for Answer Set Programming (ASP) named LLASP was proposed.

LLASP was specifically trained to encode fundamental ASP program patterns by using an ad-hoc dataset covering a wide variety of fundamental problem specifications in ASP.

LLASP proved to perform significantly better in generating correct ASP programs than larger and more powerful LLMs, especially regarding the semantic perspective.

3.3.4.3 Transformers (2024)

The paper “Learning Semantic Structure through First-Order-Logic Translation” [CA24] shows that language models sometimes struggle with predicate argument structure, confusing which predicates to apply to which objects.

Recovering predicate argument structure is fundamental to capturing and reasoning about the semantics of natural language sentences.

For this reason, to mitigate these problems, two tasks (question answers, Q/A, and FOL translation) and two regimes (prompting and fine-tuning) are explored.

In FOL translation, several large language models were fine-tuned on synthetic datasets designed to gauge their generalization abilities. In Q/A instead, encoder models (e.g., BERT and RoBERTa) were fine-tuned and prompting for LLMs.

The results showed that FOL translation for LLMs is better suited to learn predicate argument structure than Q/A. In fact, the FOL translation task makes hallucinatory content immediately obvious and also completely determines predicate argument structure.

3.3.5 Datasets and benchmarks

3.3.5.1 LogicNLI (2021)

The paper “Diagnosing the First-Order Logical Reasoning Ability Through LogicNLI” [TLC⁺21] proposes a systematic diagnostic method for first-order logical (FOL) reasoning by introducing a benchmark named Logical Natural Language Inference (LogicNLI) ³.

The dataset disentangles the FOL reasoning from commonsense inference and can be used to diagnose language models (LMs) from four perspectives: accuracy, robustness, generalization, and traceability. Experiments on three LMs (BERT, RoBERTa, and XLNet), have uncovered the weaknesses of these language models on FOL reasoning.

³<https://huggingface.co/datasets/tasksource/LogicNLI>

3.3.5.2 GSM8K-Prolog (2023)

In “Exploring an LM to generate Prolog Predicates from Mathematics Questions” [YT23], the dataset GSM8K-Prolog ⁴ is introduced to train a LLaMA-7B model to generate Prolog predicates from mathematics questions.

GSM8K-Prolog is a Prolog annotated version of the GSM8K math reasoning dataset, containing the instruction for the task the model should perform (in this case, instruct the model to use Prolog to solve math questions), the input (the question expressed in natural language to a grade school math problem) and the output (the Prolog program which solves the math question).

3.3.5.3 FOLIO (2024)

The paper “FOLIO: Natural Language Reasoning with First-Order Logic” presents FOLIO ⁵, a human-annotated, logically complex, and diverse dataset for reasoning in natural language, equipped with FOL annotations.

FOLIO consists of 1430 unique conclusions, each paired with one of 487 sets of premises used to deductively reason for the validity of each conclusion. The results of experiments on FOLIO show that a subset of the dataset present a challenge for the LLM GPT-4.

3.4 Analysis

What emerges from the relate works analysis is that there are different methodologies and tools available for translating from natural language to a formal representation based on logic.

The ambiguity of natural language is one of the most commented problems for the task. For this reason, many papers relied on the use of a controlled version of English or any other form of intermediate representation of language to have both a machine and human understandable language.

Older papers focus on more symbolic approaches especially using grammars, while newer papers focus on the creation of databases for training neural models.

On the one hand, symbolic approaches involve complex translation mechanisms that might be less immediate to implement, although once implemented they tend to result in portable

⁴<https://huggingface.co/datasets/Thomas-X-Yang/gsm8k-prolog>

⁵<https://huggingface.co/datasets/yale-nlp/FOLIO>

programs. They are also more limited in terms of input flexibility, often requiring a controlled language input.

On the other hand, neural approaches typically use LLMs to disambiguate the original NL input, which notably tend to struggle in logic reasoning tasks, demand substantial computational resources in terms of processing time and memory, and require huge amounts of training data.

Most of the datasets mentioned did not focus on Prolog representation, being mainly used for FOL expressions.

Although translating from FOL (or any other logic representation based on FOL) to Prolog is feasible, the dataset we needed had to be a synthesis of shared Prolog programming practices by common users. Hence a formal translation would possibly not capture this aspect as well as a dataset based on actual Prolog code.

Therefore, GSM8K-Prolog was the main dataset considered before deciding to create one ad hoc for this thesis. The exclusion was motivated by the lack of direct correspondence between the sentences in NL in the text and the Prolog representation in GSM8k-Prolog, which was fundamental for our purposes for this thesis.

Chapter 4

Dataset Creation

A requirement for the thesis was having available a dataset containing Prolog facts, rules, and queries and their corresponding sentences expressed in natural language.

This dataset needed to be as diverse as possible, since it would be used for collecting in an agnostic way, practices that users commonly adopt to translate concepts expressed in natural language into Prolog. It would be also used as a benchmark to test both our tool and GPT translation capabilities.

To our knowledge, there has not been a dataset with these characteristics yet available. Hence we decided to create it manually using online resources, which consisted mostly of tutorials on how to use Prolog for beginners, as well as manuals for the language. All resource links are specified in the dataset.

The result was a structured dataset containing Prolog expressions and at least one possible meaning expressed in English for each of them.

The dataset is freely available to the research community¹ and consists of seventeen sheets, featuring a total of 613 facts, 55 queries and 38 rules from web resources or generated by GPT. It also contains further 48 running examples and 30 semantic tests (which include facts, rules, and queries).

1. Facts Websites: contains 79 Prolog facts with all ground terms and corresponding meaning in English, both from online resources.
2. Facts Mix: contains 257 Prolog facts and the corresponding meaning in English. The former comes from online resources, while the latter comes from GPT-5.

¹Dataset link, accessed on March 10, 2026.

3. Facts Variables: contains 3 Prolog facts which contain at least one variable and the corresponding meaning in English, both from online resources.
4. Queries Websites: contains 34 Prolog queries and corresponding meaning in English, both from online resources.
5. Queries Mix: contains 21 Prolog queries and the corresponding meaning in English. The former comes from online resources, while the latter comes from GPT-5.
6. Rules Websites: contains 17 Prolog rules and corresponding meaning in English, both from online resources.
7. Rules Mix: contains 21 Prolog rules and the corresponding meaning in English. The former comes from online resources, while the latter comes from GPT-5.
8. 0-arity facts (GPT): contains 79 arity zero facts where both the Prolog expression and corresponding meaning in English were generated by GPT-5.
9. 1-arity facts (GPT): contains 98 arity one facts where GPT-5 generated both the Prolog expression and the corresponding meaning in English.
10. 2-arity facts (GPT): contains 97 arity two facts in which both the Prolog expression and the corresponding meaning in English were generated by GPT-5.
11. Running examples: Further 48 running examples handcrafted by us, still faithfully respecting the observed practices. 15 of them were submitted to Prolog users and have an additional column which specifies their “agreement score”, which is the percentage of experts who agreed with the representation. There are two columns for the Prolog representations, default and custom. The default refers to the results obtained using the default settings of the tool Talk2Prolog, while custom allows to set flags to adapt the output according to the user’s preference. The agreement score was calculated based on the default version.
12. Semantic examples: Further 30 examples handcrafted by me (in total 104, Prolog expressions), focusing on the sentences and expected results. They were created to test the capabilities of the tools to convey the meaning of the sentences they translate.

We decided to use GPT because it is a LLM trained on a massive amount of data accessible on the internet. Therefore, it is a powerful synthesis tool by antonomasia of all domain that can be described in digital form. In our scenario, it statistically reproduces the most likely way to represent a sentence in Prolog and vice versa. Hence, delegating the completion of missing couples in the dataset to GPT-5, is coherent with the agnostic representation of shared practices.

Inconsistencies between the Prolog representation and the corresponding English meaning, and small grammar mistakes found in the online resources listed in the dataset were corrected.

The arity of all terms is always specified for facts and queries (including nested ones for the facts), and the number of variables (including anonymous variables) is specified for the rules and queries.

4.1 Sources for the dataset

Here is the list of principal resources used for the construction of the Prolog dataset, chosen due to their many Prolog examples that could be paired with a simple English sentence:

- GPT 5.2
- Athena
- Aberystwyth
- Tutorials Point
- SFU
- CS Toronto
- CS Trincoll
- Learn Prolog Now!
- Nosco
- Kiddle encyclopedia
- GeeksforGeeks
- Built-in

Some are Prolog resources from the department of Computer Science of universities, some are more informal tutorials on Prolog.

4.2 Selected examples

4.2.1 Facts Websites

This sheet contains Prolog facts and English meanings from web resources. There can be nested facts, but not variables, and the arity range from 0 to 2.

Some examples are reported in Tab. 4.1.

Prolog	English	Arity
foggy.	it is foggy	0
dinner(pizza).	pizza is a dinner	1
age(john,32).	John is 32 years old	2
capital_of(france, paris).	Paris is the capital of France	2
not(likes(john,pizza)).	John does not like pizza	1;2

Table 4.1: Facts Websites sheet.

4.2.2 Facts Mix

This sheet contains Prolog facts from web resources and English meanings generated by GPT 5.2. There can be nested facts, but not variables, and the arity ranges from 0 to 2.

Some examples are reported in Tab. 4.2.

Prolog	English	Arity
party.	There is a party.	0
can_cook(priya).	Priya can cook.	1
mother(queen_elizabeth_II, prince_charles).	Queen Elizabeth II is the mother of Prince Charles.	2
population(germany, 83251851).	Germany has a population of 83,251,851 people.	2
consequence(creepage_path, 'replace insulator').	A creepage path leads to replacing the insulator.	2

Table 4.2: Facts Mix sheet.

4.2.3 Facts Variables

This sheet contains Prolog facts and English meanings from web resources. In this case, the facts might contain variables. In addition to the arity, the number of variables is also reported.

Some examples are reported in Tab. 4.3.

Prolog	English	Arity	Variables
parent(X).	Everyone is a parent!	1	1
likes(X, susie).	Everyone likes Susie	2	1
likes(john, Y).	John likes everybody	2	1

Table 4.3: Facts Variables sheet.

4.2.4 Queries Websites

This sheet contains Prolog queries and English meanings from web resources. These queries can be composed of more goals. Both the arity, which can range from 0 to 2, and the number of variables are indicated.

Some examples are reported in Tab. 4.4.

Prolog	English	Arity	Variables
?- dinner(sandwich).	Is sandwich a dinner?	1	0
?- studies(charlie, What).	What does charlie study?	2	1
?- owns(jane, X), sedan(X).	Does jane own sedan?	2;1	1
?- loves_to_eat(kunal,pasta).	Does Kunal love to eat pasta?	2	0

Table 4.4: Queries Websites sheet.

4.2.5 Queries Mix

This sheet contains Prolog queries from web resources and English meanings generated by GPT 5.2. These queries can be composed of more goals. Both the arity, which can range from 0 to 2, and the number of variables are indicated.

Some examples are reported in Tab. 4.5.

Prolog	English	Arity	Variables
?- herbivore(butterfly).	Is the butterfly a herbivore?	1	0
?- herbivore(X).	Which animals are herbivores?	1	1
?- pyramid(fox).	What level is the fox in the food pyramid?	1	0
?- friend(harry, X).	Who is a friend of Harry?	2	1

Table 4.5: Queries Mix sheet.

4.2.6 Rules Websites

This sheet contains Prolog rules and English meanings from web resources. The number of variables for each rule is indicated.

Some examples are reported in Tab. 4.6.

Prolog	English	Variables
meal(X) :- food(X).	Every food is a meal	1
professor(X, Y) :- teaches(X, C), studies(Y, C).	X is a professor of Y if X teaches C and Y studies C.	3
friends(jack, bili) :- lovesCricket(jack), lovesCricket(bili).	Jack and Bili are friends if both of them love to play cricket.	0
enemies(X,Y) :- not(likes(X,Y)),not(likes(Y,X)).	X and Y are enemies if they don't like each other	2

Table 4.6: Rules Websites sheet.

4.2.7 Rules Mix

This sheet contains Prolog rules from web resources and English meanings generated by GPT 5.2. These queries can be composed of more goals. The number of variables for each rule is indicated.

Some examples are reported in Tab. 4.7.

Prolog	English	Variables
listens_to_music(ananya) :- sing_a_song(ananya).	If Ananya sings a song, then she also listens to music.	0
herbivore(X) :- eats(X, Y), plant(Y).	X is a herbivore if X eats Y and Y is a plant.	2
omnivore(X) :- herbivore(X), carnivore(X).	X is an omnivore if X is both a herbivore and a carnivore.	1
pyramid(X, Y) :- prey(Y, X).	X is at a higher level in the food pyramid than Y if Y is prey to X.	2

Table 4.7: Rules Mix sheet.

4.2.8 0-arity facts (GPT)

This sheet contains Prolog facts generated by GPT 5.2 with arity zero.

Some examples are reported in Tab. 4.8.

Prolog	English
bright.	It is bright.
talking.	Someone is talking.
angry.	I am angry.
new_moon.	There is a new moon.

Table 4.8: 0-arity facts (GPT) sheet.

4.2.9 1-arity facts (GPT)

This sheet contains Prolog facts generated by GPT 5.2 with arity one.

Some examples are reported in Tab. 4.9.

Prolog	English
asleep(vincent).	Vincent is asleep.
artist(sophia).	Sophia is an artist.
yellow(sunflower).	The sunflower is yellow.
closed_source(program).	The program is closed source.

Table 4.9: 1-arity facts (GPT) sheet.

4.2.10 2-arity facts (GPT)

This sheet contains Prolog facts generated by GPT 5.2 with arity two.

Some examples are reported in Tab. 4.10.

Prolog	English
has(bob, car).	Bob has a car.
helps(diana, alice).	Diana helps Alice.
works_in(charlie, office).	Charlie works in an office.
watches(diana, documentary).	Diana watches a documentary.

Table 4.10: 2-arity facts (GPT) sheet.

4.2.11 Running examples

In this section, we will discuss the feedback provided by a survey we submitted to Prolog users about our methodology.

We submitted 15 of our running examples (see Tab. 4.11) to Prolog experts, average users, and new learners to have the opportunity to receive their opinions about the methodology implemented by our tool.

This allows us to individuate possible improvements by evaluating how much our methodology mirrors programming practices actual Prolog users follow.

We received 33 answers in total, one for each participant.

The participants had the following Prolog skills:

- 15 (45%) were very familiar with Prolog.
- 8 (24%) were almost familiar with prolog.
- 6 (18%) followed some lessons on Prolog.
- 4 (12%) did not know Prolog, but read the instructions provided by us.

Regarding the professional position of the participants in the moment the survey was made, some of them described themselves with more than one category. Therefore, the percentages reported below are calculated based on 38 answers:

- 15 (40%) worked in academia with a tenure track position.
- 3 (8%) worked in academia with a non permanent position (e.g. research scholarship, research assistant).
- 3 (8%) were PhD students.
- 12 (32%) were Master of Science students.
- 5 (13%) were from people who work in a company.

The main observations provided by the survey participants regarded the following aspects.

4.2.11.1 Time and space

Some participants suggested the possibility of contextualizing Prolog expressions with additional information, such as time and space.

For example, for the Prolog expression `late.` (sentence 1), adding a time (or time interval) *T* or a constant, such as *now*, that could act as a reference point, or a relationship between two times.

Regarding the Prolog expressions related to places instead, such as `library.` (sentence 3) and `?- bookshop.` (sentence 4), some suggested the addition of a reference to indicate where exactly these places are located.

Sentence	Prolog representation	Agreement score
1. It's late.	late.	73 %
2. It's an interesting topic.	interesting(topic).	85 %
3. There is a library.	library.	67 %
4. Is there a bookshop?	?- bookshop.	61 %
5. The book is excellent if Isaac Asimov wrote it.	excellent(book) :- writes(isaac_asimov, book).	67 %
6. Are authors satisfied?	?- satisfied(authors).	61 %
7. Charlie is happy if Alice and Bob enjoy the book.	happy(charlie) :- enjoys(alice, book) , enjoys(bob, book).	82 %
8. Alice lives in London.	lives_in(alice, london).	97 %
9. If Alice and Bob hate the book, then Charlie is sad.	sad(charlie) :- hates(alice, book) , hates(bob, book).	85 %
10. If Bob gets a good grade or learns a new topic, then he is happy.	happy(bob) :- gets(bob, good(grade)) ; learns(bob, new(topic)).	67 %
11. The canvas is being painted by Bob.	paints(bob, canvas).	85 %
12. Alice does not like cupcakes.	not(likes(alice, cupcakes)).	82 %
13. Who paints the canvas?	?- paints(Who, canvas).	85 %
14. Famous musicians sell one thousand albums.	sell(famous(musicians), amount(albums, 1000))	55 %
15. What does Bob paint?	?- paints(bob, What).	88 %

Table 4.11: Natural language sentences proposed in the survey alongside their Prolog representations, as per the methodology, and corresponding agreement score based on the survey participants' evaluations.

4.2.11.2 Pronouns substitution

Some comments were made about the decision of not substituting pronouns with their corresponding nouns in certain cases.

For example, the Prolog expression `interesting(topic).` (sentence 2), the pronoun “it” from the original sentence is not included in the translation, since it does not refer to anything mentioned previously in the sentence.

4.2.11.3 Predicates acting as types

Another common suggestion was the use of atoms that represented nouns in the translation as predicates instead, in order to create a sort of type system.

For example, the predicates `library` (sentence 3) and `bookshop` (sentence 4), could be used to wrap an identifier of a specific library or bookshop.

4.2.11.4 Use of variables

Many suggested that common nouns could be represented as variables in the Prolog representation, since they are normally used to represent an entire category instead of a specific entity of a category.

For example, the Prolog expression `excellent(book):- writes(isaac_asimov, book).` (sentence 5), the atom `book` could be represented as a variable `Book` to make a general statement about the quality of the books that have been written by Isaac Asimov.

4.2.11.5 Two rules instead of one

Some participants prefer using two separate rules to model disjunction in Prolog instead of a single one. Their motivations were for reasons of clarity, maintainability, and understandability.

For example, the Prolog expression `happy(bob):- gets(bob, good(grade)); learns(bob, new(topic)).` (sentence 10), could be separated into two rules:

- `happy(bob):- gets(bob, good(grade)).`
- `happy(bob):- learns(bob, new(topic)).`

4.2.11.6 Negation

Some of the participants preferred the use of predicates that already act as negation of a certain concept instead of the nesting with the `not` predicate.

For example, the Prolog expression `not(likes(alice, cupcakes)).`, instead of negating the positive predicate `likes`, using a special predicate for this purpose, such as `dislikes` or `not_likes`.

4.2.11.7 Additional Prolog constructions

Some of the comments regarded the inclusion of other Prolog constructs, such as the `forall` predicate and comparison operators.

For example, the Prolog expression `?- satisfied(authors).` (sentence 6) could be rewritten using the `forall`, since the sentence “Are authors satisfied?” is universally quantified in a implicit way.

Another example is the Prolog expression `sell(famous(musicians), amount(albums, 1000))..` (sentence 14) could be better expressed using the “greater than” operator, represented in Prolog as `>`, since the sentence “Famous musicians sell one thousand albums.” implicitly assumes that if a musician sells more than 1000 albums, they can still be considered famous.

4.2.11.8 Conclusions on the survey

Many of the observations were related to the fact that the methodology is focused on modeling only what is explicitly included in the sentence. Therefore, no additional information is included.

This is due to the fact that the only requirement for the methodology to work is the sentence being grammatically correct, but it does not need to necessarily provide contextual clarity.

This allows the methodology to be more flexible in the case of out-of-context sentences, and easy to implement, but at the same time, the result might be considered ambiguous depending on the context, and it is more rigid in terms of not being able to represent some universal sentences.

4.2.12 Semantic examples

This sheet contains sentences and possible Prolog translations handcrafted by me (which are not necessarily in the same way Talk2Prolog would represent them).

The focus of this sheet is not on the representations from a syntactic point of view, but in the information they convey. Hence, the sentences are divided in blocks, one for each semantic test, and are ordered inside the block. Expected answers for the interrogative questions are also reported, to check if the corresponding query is able to retrieve the meaning of the preceding sentences.

Some examples are reported in Tab. 4.12.

Sentences	Result	Prolog
The streets are wet if it is raining.		wet(streets) :- raining.
It is raining.		raining.
Are the streets wet?	TRUE	?- wet(streets).
Alice reads scientific articles if she is a researcher.		reads(alice, scientific(articles)) :- researcher(alice).
Does Alice read scientific articles?	FALSE	?- reads(alice, scientific(articles)).
What does Alice read?	FALSE	?- reads(alice, What).
If Charlie is free and feels well, then he plays tennis.		plays(charlie, tennis) :- free(charlie) , feels(charlie, well).
Charlie feels well.		feels(charlie, well).
Does Charlie play tennis?	FALSE	?- plays(charlie, tennis).
Charlie is sad if Alice or Bob are mean.		sad(charlie) :- mean(alice) ; mean(bob).
Is Charlie sad?	FALSE	?- sad(charlie).
Peter stops the train.		stops(peter, train).
The train is stopped by Peter.		stops(peter, train).
What is stopped by Peter?	train, train	?- stops(peter, What).
Charlie does not speak French.		not(speaks(charlie, french)).
Does not Charlie speak French?	TRUE	?- not(speaks(charlie, french)).

Table 4.12: Semantic examples sheet.

Chapter 5

Practices-driven Methodology and Design

In this section, we introduce a methodology that describes a systematic and reproducible procedure to translate a sentence expressed in formal, simplified English into a plausible and practice-compliant Prolog representation, which could be a fact, a rule, or a query.

By plausible, we mean that the *semantic* information of the original sentence is retained as much as possible in the resulting Prolog expression, allowing the user to query and assert predicates without compromising consistency, real-world applicability, and human interpretability. This fulfills the necessity to infer new information based on already given knowledge.

By practice-driven instead, we mean that the *syntax* of the representation is not only coherent with Prolog syntax rules, but also based on common Prolog practices.

In fact, we created this methodology as a way to establish a set of rules that standardize the practices often adopted informally by Prolog users to express information in the programming language.

These rules were inferred using a bottom-up approach by analyzing the dataset.

The objective of the methodology is to extract all the predicates and arguments necessary to form a Prolog term by using a rule-based approach based on English grammar.

The motivation in creating such a methodology was to use it as a guide, agreed upon by many Prolog users for implementing translators from natural language to Prolog. Whatever the technology behind the different implemented translators, if they stick to the methodology, their translations should be identical, hence paving the way to semantic interoperability of the different Prolog knowledge bases.

For the scope of this thesis, we used the methodology as a theoretical basis to model a rule-based implementation and to engineer LLM prompts for generating Prolog rule, in order to compare both approaches. The latter is generally more expensive in computational terms, such as time and space complexity, and heavily relies on deep-learning techniques.

For this reason, we focused on aspects our implementation could realistically handle given the time available for its development. This affected some of the choices we made for the translations suggested by the methodology.

In particular, we limited our problems to the following cases:

- We are focusing on sentences in the present tense, although the methodology could be applied in the same way for the past simple.
- Both definite and indefinite articles were discarded from the final result, since they are not common in Prolog representations.
- We only considered resulting terms with an arity ranging from zero to two.
- We are not considering sentences containing modal verbs, since they would require modal logic to be represented, and this type of logic is not inherently supported by classic Prolog.
- Similarly to the previous point, we do not consider time, unless explicitly expressed in the sentence, since it would require temporal logic to be represented. Again, this type of logic is not inherently supported by classic Prolog.

Due to our background, the main use case we considered is the possibility of translating natural language sentences prompted by a user to Prolog expressions that could be used inside a multi-agent systems, such as BDI programs implemented in Jason.

Considering a belief-desire-intention (BDI) architecture, facts and rules could represent beliefs. This focus influenced certain decisions over others for the approach taken in the methodology translation.

In this context, the default is not using any sort of temporal logic to model time, since the information is considered to be continuously updated when needed, being added when new or removed when not relevant anymore.

To create the methodology, we considered the different types of sentence structures based on the number and type of clauses, which we are going to discuss in the following sections.

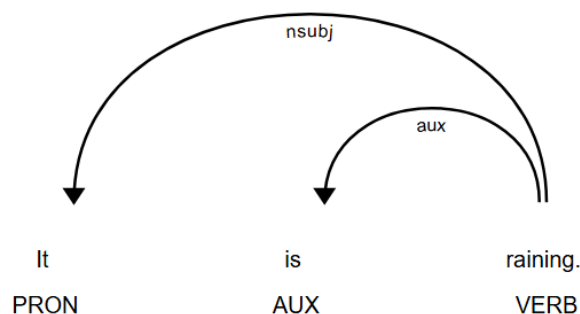


Figure 5.1: Expletive construction: It is raining.

5.1 Simple sentences

Simple sentences contain just one independent clause. This clause is represented as a Prolog fact, whose arity and arguments depend on the clause pattern type.

In this section, we are mostly considering simple affirmative sentences in the active voice and focusing on the most external term, which will become the predicate, but we will discuss the details of how the methodology handles negative simple sentences, the passive voice, and the necessity of nesting as well.

5.1.1 Expletive constructions

Expletive constructions refer to clauses that have “dummy” subjects, such as “it” or “there”. These words do not refer to anything, being semantically impersonal, and exist only to satisfy a syntactic requirement.

A very common example of the use of a dummy subject is when the word “it” is used to describe the weather. An example is the sentence “It is raining”. See Fig. 5.1.

Another way in which the word “it” can be used as a dummy subject is in extraposition constructions, but this type of construction was not addressed by us.

The word “there” instead is used as a dummy subject in existential clauses. An example is the sentence “There is a library”. See Fig. 5.2.

A clause containing an expletive construction is represented in Prolog only by its complement, which follows the copular verb “to be”. Therefore, the arity of the resulting predicate depends on the complement and can range from 0 to 2.

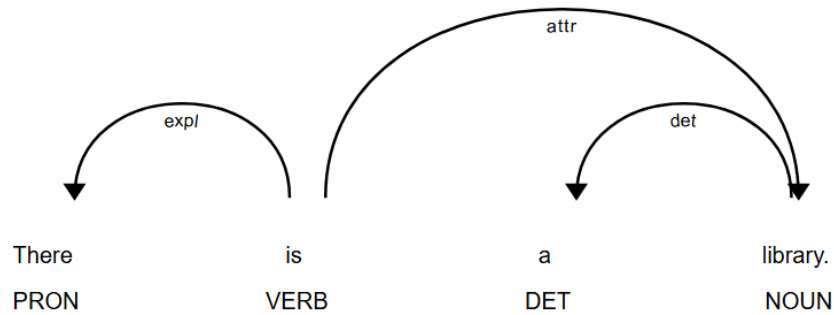


Figure 5.2: Expletive construction: There is a library.

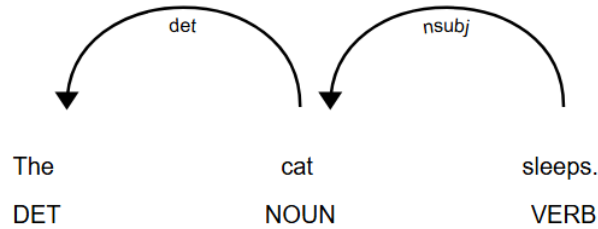


Figure 5.3: SV: The cat sleeps.

5.1.2 SV clauses

This clause pattern consists of a subject (S) and an intransitive verb (V).

A clause of this type is represented as a 1-arity term, where the term name is the lexical verb, and the argument of the term is the subject.

For example, if we consider the sentence “The cat sleeps.”, the verb “sleeps” becomes the term name, and the subject “cat” becomes the term argument, resulting in the Prolog expression `sleeps(cat)`. See Fig.5.3.

5.1.3 SVA clauses

This clause pattern consists of a subject (S), a copular verb (V), and an adverbial (A).

A clause of this type is represented as a 2-arity term, where the predicate is the lexical verb, the first argument is the subject, and the second argument is extracted from the adverbial.

The adverbial might be just a word, the adverb, in this case it will be considered entirely as the second argument, or an adverbial phrase. In the latter case, the last adverb appearing

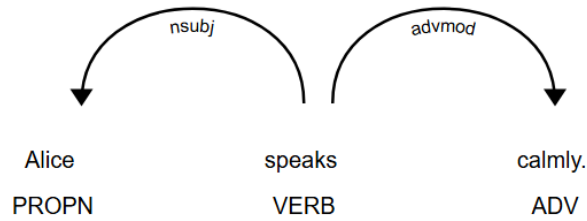


Figure 5.4: SVA: Alice speaks calmly.

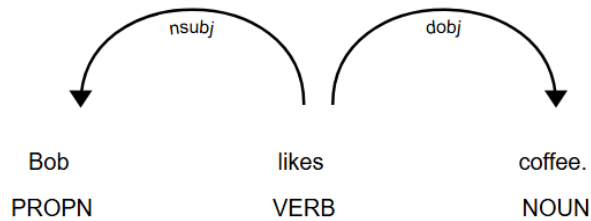


Figure 5.5: SVO: Bob likes coffee.

in the phrase will be considered.

For example, if we consider the sentence “Alice speaks calmly.”, the verb “speaks” becomes the term name, the subject “Alice” becomes the first argument, and the adverb calmly becomes the second argument, resulting in the Prolog expression `speaks(alice, calmly)`. See Fig.5.4.

If instead the sentence was “Alice speaks very calmly”, the degree adverb “very” inside the adverbial phrase would be ignored, resulting in the same Prolog expression mentioned before.

5.1.4 SVO clauses

This clause pattern consists of a subject (S), a monotransitive verb (V) and a direct object (O).

A clause of this type is represented as a 2-arity term where the predicate is the lexical verb, the first argument is the subject and the second argument is the direct object.

For example, if we consider the sentence “Bob likes coffee.”, the verb “likes” becomes the term name, the subject “Bob” becomes the first argument and the object “coffee” becomes the second argument, resulting in the Prolog expression `likes(bob, coffee)`. See Fig. 5.5.

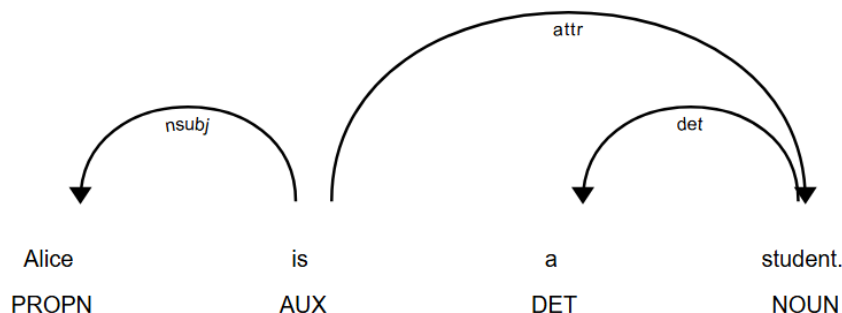


Figure 5.6: SVC: Alice is a student.

5.1.5 SVC clauses

This clause pattern consists of a subject (S), a copular verb (V) and a subject complement (C).

When the verb “to be” is the copula, the sentence is represented as a 1-arity term, where the predicate is the complement, and the argument is the subject.

For example, if we consider the sentence “Alice is a student.”, the complement “student” becomes the term name, the subject “Alice” becomes the term argument, and the copular verb

is ignored, resulting in the Prolog expression `student(alice)`. See Fig.5.6.

5.1.6 Simple negative sentences

Negative sentences are handled by negating the term corresponding to the affirmative version of the sentence using the “not” predicate (in an actual implementation, a predicate used should be created on purpose to represent the negation, since the predicate “not” is a built-in predicate in Prolog for the weak negation).

For example, if we consider the sentence “Alice does not like cupcakes.”, we first transform the affirmative version of the sentence, “Alice likes cupcakes.”, into its Prolog representation, which in this case is `likes(alice, cupcakes)`., and then negative it by using the “not” predicate. As a result, we get `not(likes(alice, cupcakes))`.. See Fig. 5.7.

Semantically speaking, it does not act as an actual negation in the traditional natural language sense. In the example seen before, `not(likes(alice, cupcakes))`. and `likes(alice, cupcakes)`. are not opposite, contradictory facts. They both exist on their own, as simple representations of sentences independently of their meaning.

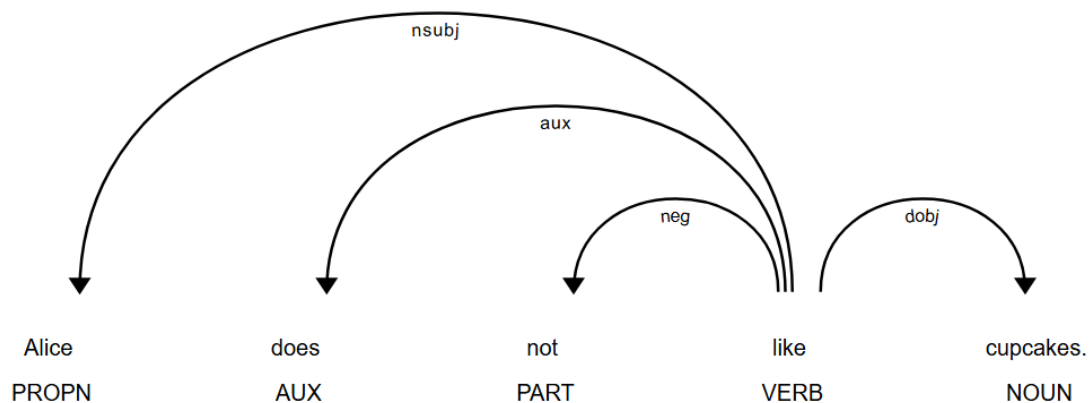


Figure 5.7: Negative: Alice does not like cupcakes.

The same principle applies to words with negative prefixes. The word “dislike” does not contradict the word “like”.

From a Prolog point of view, this corresponds to the principle of negation as failure: to negate a sentence, it is sufficient to not represent it.

5.1.7 Simple passive sentences

Although the emphasis shifts on the agent and patient accordingly if we are using the active or passive voices in the sentences, the semantic meaning is substantially the same, so we decided to translate passive sentences as if they were active. Therefore, the subject of the sentence is the agent and the object the patient.

For example, the passive sentence “The canvas is being painted by Bob.” corresponds to the active sentence “Bob paints the canvas.”, so it will be represented in Prolog in the same way as the active version would be represented, so with the expression `paints(bob, canvas)`. See Fig 5.9 and Fig 5.8, for the passive and active versions of the same concept, respectively.

5.1.8 Simple interrogative sentences

Interrogative sentences are questions, which are represented in Prolog as queries and translated similarly to affirmative sentences.

For the methodology, we are considering only direct questions. This type of questions does not use collocations regarded as politeness that do not convey any information about the question itself, such as “Could you please tell me...”.

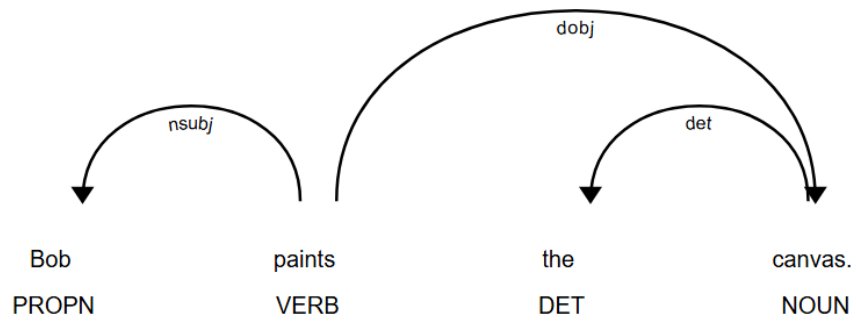


Figure 5.8: Active: Bob paints the canvas.

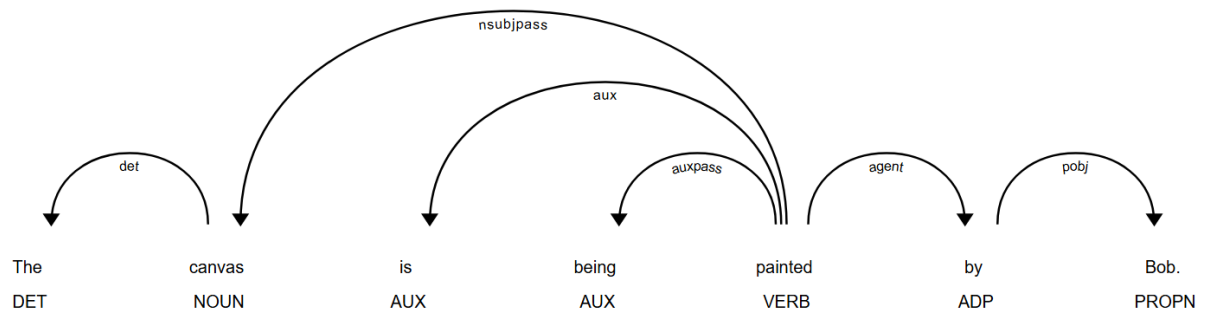


Figure 5.9: Passive: The canvas is being painted by Bob.

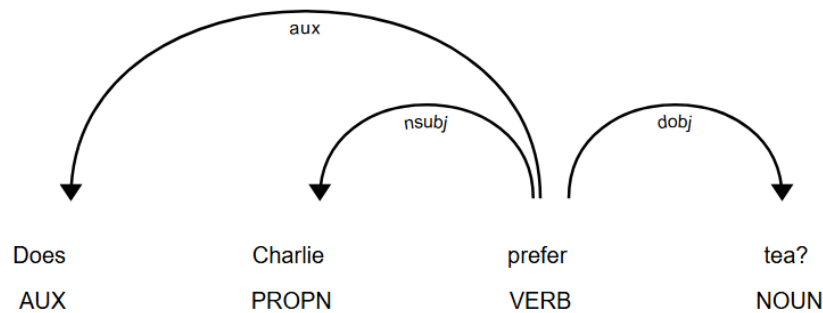


Figure 5.10: Yes/no question: Does Charlie prefer tea?

The simplest form of interrogative sentences are yes/no questions. This type of questions uses subject-verb inversion, such as in the sentence “Is it cold?”, or auxiliary verbs, such as the verb “do” in the sentence “Does Charlie prefer tea?”. See Fig. 5.10

We translate the question as if it were an affirmative sentence by ignoring the inversion and auxiliary verbs. To tell Prolog it is a query, the resulting term is prefixed by the symbol “?-”.

For example, if we consider the sentence “Is it cold?”, we represent the affirmative version of the sentence “It is cold.” as `cold.` and the interrogative version as `?- cold..`

If we consider instead the sentence “Does Charlie prefer tea?”, we represent the affirmative version of the sentence “Charlie prefers tea.” as `prefers(charlie, tea).` and the interrogative version as `?- prefers(charlie, tea)..`

Another type of interrogative sentences are “Wh-” questions. They start with a question word such as. Questions words are always represented as variables. This kind of interrogative sentence can be divided into subject and object questions.

In subject questions, the question word is the subject of the sentence. This kind of interrogative question is treated exactly in the same way as affirmative questions, since it is not inverted.

For example, if we consider the sentence “Who paints the canvas?”, the question word “who” is the subject. We represent the affirmative version of the sentence “Who paints the canvas.” as `paints(Who, canvas).` and the interrogative version as `?- paints(Who, canvas)..` See. Fig. 5.11.

In object questions, the question word is the complement object of the sentence. This kind of interrogative question uses inversion and the auxiliary verb “do”, which are both ignored, exactly like in the case of yes/no questions.

For example, if we consider the sentence “What does Bob paint?”, “What” is the com-

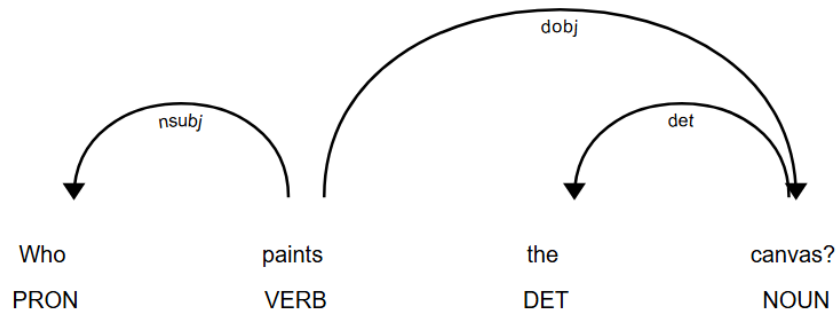


Figure 5.11: Subject question: Who paints the canvas?

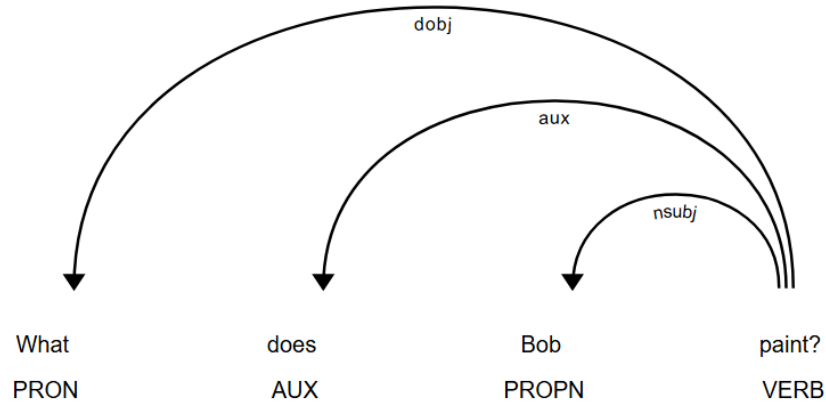


Figure 5.12: Object question: What does Bob paint?

plement object (and “Bob” the subject). By ignoring the inversion and the auxiliary verb “does”, we represent the affirmative version of the sentence “Bob paints what.”. We represent the affirmative version as `paints(bob, what)`. and the interrogative version as `?- paints(bob, what)`.. See Fig. 5.12.

We did not consider alternative questions, which indicate the possible choices of answer (for example, the sentence “Does Charlie prefers tea or coffee?”).

We also did not consider tag questions, which are affirmative sentences turned into questions by adding a question tag at the end (for example, the sentence “Charlie is coming to the lesson, isn’t he?”).

5.2 Complex sentences

Complex sentences contain one independent clause and one dependent clause (also known as a subordinate clause) linked by a subordinating conjunction.

In this methodology, we only consider complex sentences that use the word “if” as a subordinating conjunction.

In particular, we are considering zero conditional sentences, where the tense of both clauses is the simple present and are used to refer to general truths.

This clause is represented as a Prolog rule, where the independent clause corresponds to the head of the rule, and the dependent clause to the body.

Since rules are Horn clauses, only the dependent clauses can contain coordinating conjunctions for logical connectives such as “and” and “or”. These conjunctions are represented in Prolog, respectively, with the symbols “,” and “;”.

In this methodology, we only consider dependent clauses with zero or one coordinating conjunction, which must be “and” or “or”.

The arity and arguments of the head and body terms depend on the subclauses pattern types.

5.2.1 Independent clause

The independent clause, which correspond to the conclusion, is represented in Prolog following the same rules applied previously to simple sentences to form the head of the Prolog rule.

5.2.2 Dependent clause

The subordinating conjunction “if” is represented as the logical implication symbol “:-” in Prolog.

The dependent clause might contain compound elements, such as:

1. Compound subject: When two or more nouns, pronouns, or noun phrases act together as the subject of the clause, usually connected by a coordinating conjunction. For example, in the sentence “If Alice and Bob hate the book, then Charlie is sad.”, “Alice and Bob” act as the subject of the dependent clause.

2. Compound predicate: When the predicate of the clause has two or more verbs or verb phrases connected by a coordinating conjunction. For example, in the sentence “If Bob gets a good grade or learns a new topic, then he is happy.”, there are two verbs, “gets” and “learns”, connected by the coordinating conjunction “and”.
3. Compound subject complements: When two or more predicate nouns, pronouns, or adjectives appear after a linking verb. For example, in the sentence “Alice is sad if it is dark and cloudy.”, there are two adjectives in the dependent clause, “dark” and “cloudy”, which follows the linking verb “is”.
4. Compound direct object: When two or more nouns receive the action of a transitive verb. For example, in the sentence “Bob is happy if he buys books and magazines.”, both “books” and “magazines” receive the action of the transitive verb “buys”.

In the case of presence of compound elements, the dependent clause is divided into more individual clauses with one subject, one verb, and, if present, one direct object or subject complement.

These clauses correspond to the premises and are processed individually as simple sentences, which must be complete in meaning and written correctly according to English grammar rules. For this reason, capitalization, punctuation and subject-verb agreement must be considered. And also all appearing pronouns must be ideally substituted with their corresponding nouns. Finally, all clauses are merged using the connectors to form the corresponding Prolog rule.

In the case of the examples listed previously:

1. “If Alice and Bob hate the book, then Charlie is sad.” is separated into the clauses “Charlie is sad.”, which is the conclusion, and “Alice hates the book.” and “Bob hates the book.”, which are the premises.
2. “If Bob gets a good grade or learns a new topic, then he is happy.” is separated into the clauses “Bob is happy.”, which is the conclusion, and “Bob gets a good grade.” and “Bob learns a new topic.”, which are the premises.
3. “Alice is sad if it is dark and cloudy.” is separated into the clauses “Alice is sad.”, which is the conclusion, and “It is dark.” and “It is cloudy.”, which are the premises.
4. “Bob is happy if he buys books and magazines.” is separated into the clauses “Bob is happy.”, which is the conclusion, and “Bob buys books.” and “Bob buys magazines.”, which are the premises.

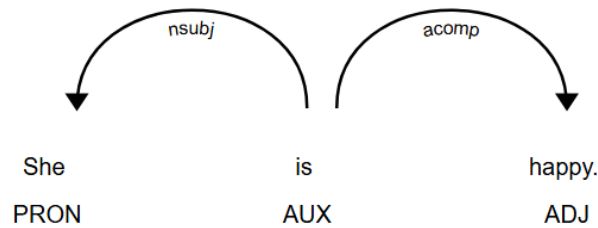


Figure 5.13: Pronoun: She is happy.

If the dependent clause does not contain compound elements, it is entirely considered the only premise.

5.3 Shared features

In this section, we discuss aspects which are common among all structures of sentences discussed previously, such as anaphora, variables, nesting, subject-verb agreement, prepositions, nouns and quantities.

5.3.1 Anaphora

Anaphoric pronouns, which are pronouns that refer to an antecedent noun, are substituted for the noun they refer to when possible, as seen in Sec. 5.2.2.

If the sentence does not have reference, the pronoun is used directly as an argument, except in the case of the pronoun “it”. In this case, the same rules seen for the expletive constructions are applied.

For example, the sentence “She is happy.” is complete, but there is not enough information to substitute the pronoun “she” with the relative noun. Therefore, by following the translation approach for SVC sentences, the corresponding Prolog representation would be `happy(she)..` See Fig. 5.13.

If we consider instead the sentence “It is an interesting topic.”, the resulting Prolog representation would be `interesting(topic).` See Fig. 5.14.

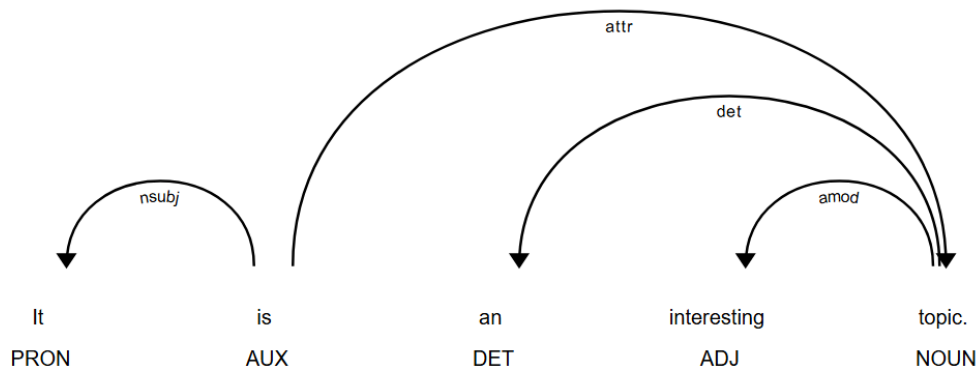


Figure 5.14: Pronoun: It is an interesting topic.

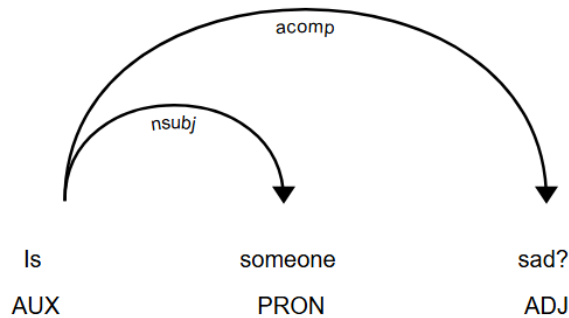


Figure 5.15: Variable: Is someone sad?

5.3.2 Variables

As discussed in Sec. 5.1.8, the question words become variables. Another type of words that become variables are a subset of indefinite pronouns that have as prefix “any”, “every” or “some”.

For example, the sentence “Is someone sad?” is represented in Prolog as `?- sad(Someone).` following the rules for interrogative sentences. See Fig. 5.15.

Single letters, except for the singular first-person pronoun “I”, become variables in the resulting representation as well.

For example, the sentence “X is smart if X is curious.” is represented in Prolog as `smart(X):- curious(X).` by following the rules for complex sentences. See Fig. 5.16.

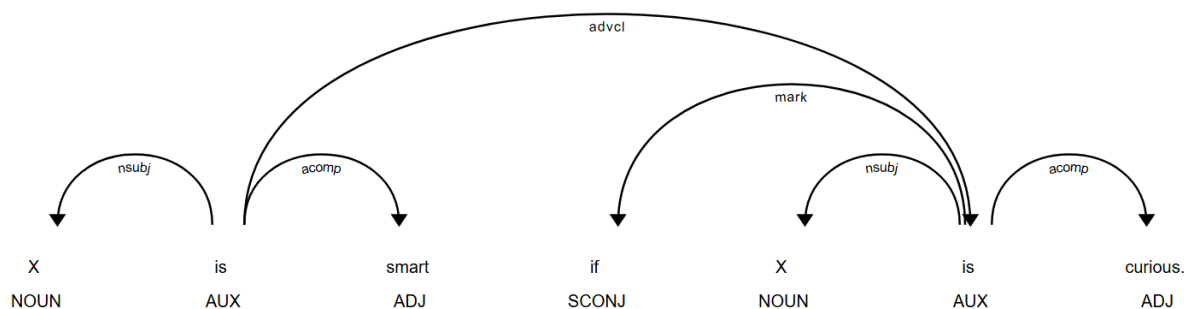


Figure 5.16: Variable: X is smart if X is curious.

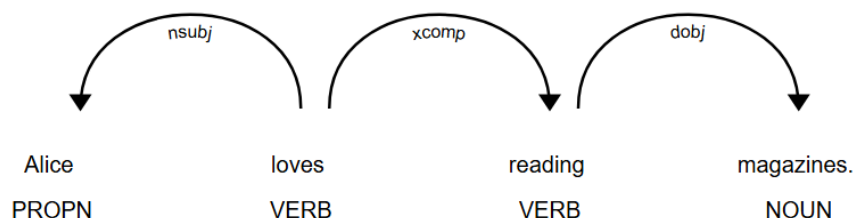


Figure 5.17: Nesting: Alice loves reading magazines.

5.3.3 Nesting

Adjectives are nested following the same order as appears in the original sentence. Using an underscore to concatenate them to the word they reference is also a common alternative, but using nesting seemed a cleaner approach, especially if in the presence of many adjectives. See Fig. 5.13.

Another situation in which nesting is used is in the case of verb patterns such as “transitive verb + infinitive” or “transitive verb + -ing”. In these situations, the second verb wraps the object of the clause without being altered.

Both verb patterns are very common for the verbs love, hate, like, and prefer.

For example, the sentence “Alice loves reading magazines” would be represented as `loves(alice, reading(magazines)).`, since “reading” is the second verb to appear and “magazines” is the object. See Fig. 5.17.

Another example is the sentence “Bob likes to play video games.”, which would be represented as `likes(bob, play(video_games)).`, since “play” is the second verb to appear and “video_games” is the object. See Fig.

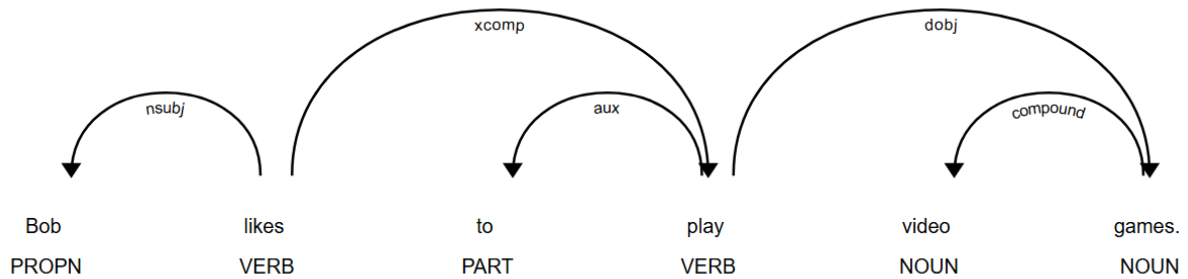


Figure 5.18: Nesting: Bob likes to play video games.

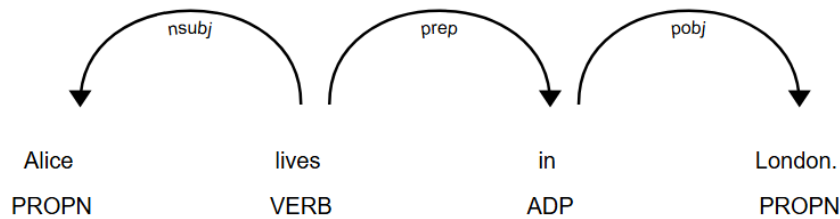


Figure 5.19: Preposition: Alice lives in London

5.3.4 Subject-verb agreement

In case the term name is a verb, it is conjugated accordingly to the subject of the sentence, which means that it agrees with the subject considering number and person.

5.3.5 Prepositions

If the verb is followed by a preposition, the preposition is appended with an underscore to the verb to form the term name.

For example, if we consider the sentence “Alice lives in London.”, the verb “lives” concatenated with the preposition “in” becomes the term name, the subject “Alice” becomes the first argument, and the object “London” becomes the second argument, resulting in the Prolog expression `lives_in(alice, london)`.. See Fig. 5.19.

5.3.6 Nouns

Nouns are represented in Prolog as atoms. In the case of compound nouns, the words are concatenated together using an underscore, as seen in Fig. 5.18.

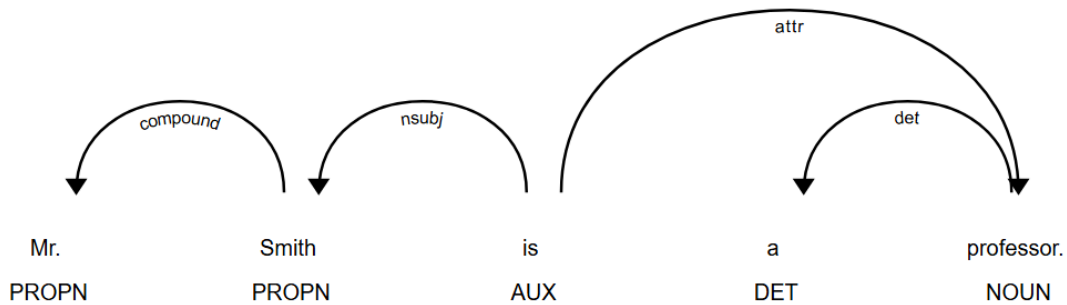


Figure 5.20: Compound noun: Mr. Smith is a professor.

If there is a dot present in the noun, such as in the case of formal titles (“Mr.”, “Ms.” and “Mrs.”), it is eliminated during the translation.

For example, in the sentence “Mr. Smith is a professor.”, the subject “Mr. Smith” is a compound noun. For this reason, following the translation rules seen for SVC clauses, the resulting Prolog representation is `professor(mr_smith)..` See Fig. 5.20.

5.3.7 Quantities

If we consider a clause whose subject is a person and the complement is a just number, we assume that the number represents an age. Therefore, the corresponding Prolog representation is a 2-arity term where the word “age” becomes the term name, the subject becomes the first argument, and the number becomes the second argument.

Regarding other quantities, we use the 2-arity predicate “quantity”, where the first argument is the object we are referencing and the second argument is the total of this object.

Numerals are preferred over words to express values. This allows for a more compact, consistent, and readable numerical representation in Prolog. It also simplifies the use of the resulting expressions in mathematical operations.

For example, if we consider the sentence “There are three friends.”, after applying the rules seen for expletive constructions in Sec. 5.1.1, we represent the word “three” using “3” and See Fig. 5.21 and use the relation “quantity” to bond this quantity to the referred object that in this case is “friends”

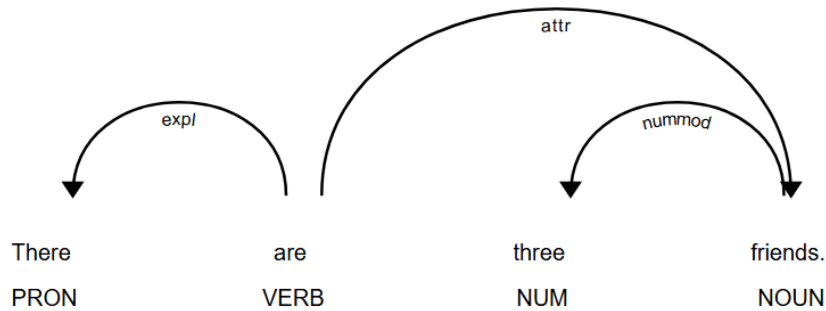


Figure 5.21: Quantity: There are three friends.

5.4 Summary

To summarize, the rules that we discovered as common practices, and that drive Talk2Prolog implementation, are:

- SV clauses become 1-arity facts of the form $V(S)$.
- SVA clauses become 2-arity facts of the form $V(S, A)$.
- SVO clauses become 2-arity facts of the form $V(S, O)$.
- SVC clauses become 2-arity facts of the form $V(S, C)$.
- Complex sentences become rules, where independent clause becomes the head and the dependent clauses the body. All clauses are made independent in order to be parsed using the above rules.
- In general, anaphoric pronouns are resolved using preceding nouns if possible, indefinite pronouns, single capital letters (except for "I") and word questions become variables, nesting is used in the presence of two verbs or adjectives, subject-verb agreement is always used, verb prepositions and compound names are concatenated using the underscore, and quantities use a predefinite predicate.
- Expletive constructions depend only on the complement to be translated, always following the rules above.

5.5 Design of Talk2Prolog

The rule-based software Talk2Prolog is a possible implementation of the methodology explained in this section.

Since we followed a declarative approach to create the translation rules, the design of Talk2Prolog was straightforward, being basically a one-to-one correspondence to the methodology, except for a few additional features implemented as flags.

Talk2Prolog receives as input a sentence and a knowledge base Prolog file, and returns as output the corresponding translation. If it is a query, it consults the file and returns the result. If it is a fact or rule, the resulting Prolog expression is inserted on the knowledge base.

Talk2Prolog is divided into three specialized components, which are orchestrated by the main component:

1. The *preprocessor* modifies the input text to make it a valid sentence in English, since it is a requirement for the methodology to be applied. It also divides the sentence into independent subsentences if the original sentence was a complex sentence, resolving anaphoras and subject-verb agreement if necessary.
2. The *semantic parser* identify the term name, arity and arguments, if any. For each subsentence, it returns the equivalent Prolog expression, which are merged by the main.
3. The *Prolog interpreter* is an component exclusive for the implementation with the purpose of interacting with the interpreter of language, in order to query the knowledge base and assert clauses.

See Fig. 5.22 for a flowchart diagram describing the high-level design of Talk2Prolog.

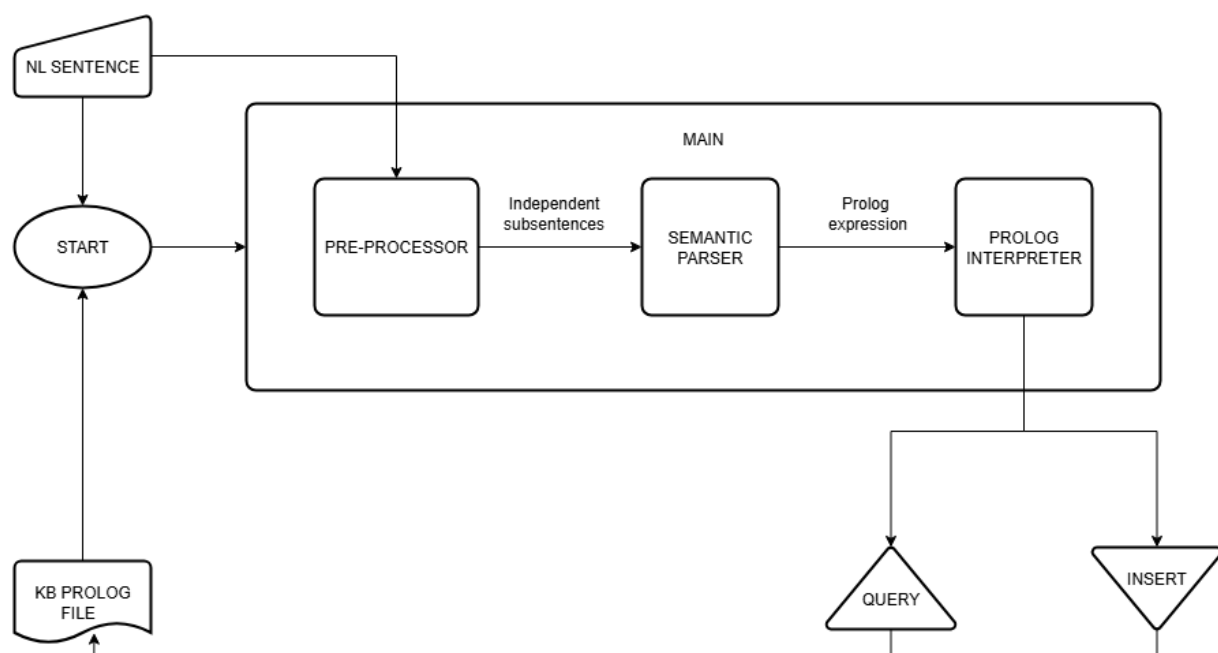


Figure 5.22: Overview of Talk2Prolog. Given as input a natural language sentence and a knowledge base Prolog file, the pre-processor normalize the sentence and gives the derived independent subsentences to the semantic parser. The semantic parser transforms the subsentences into Prolog expressions, which are then merged to be passed to the Prolog interpreter, which might query or insert a clause into the knowledge base.

Chapter 6

Implementation

Talk2Prolog, a software designed in such a way to meet the methodology described in the previous chapter, was implemented as a Python program.

The program receives as input a Prolog file, which constitutes our knowledge base, therefore the context in which we operate, and a natural-language sentence written in English.

The output of the program depends on the type of sentence given as input:

- Declarative sentences are represented as Prolog facts or rules, depending on their content, and the purpose of their translation, is to add them to the contextual, pre-existing knowledge base. The user is informed whether the assertion of the predicate in the knowledge base was successful or not.
- Interrogative sentences are represented as Prolog queries, and the purpose of their translation is to query the contextual knowledge base, created by asserting translated facts and rules, to the initial Prolog program. The query either fails, in which case the program returns false, or succeeds. In the last case, the program returns true if the term generated by the original sentence was ground, otherwise it returns the variable bindings.
- Exclamatory sentences are represented as if they were declarative sentences.
- Imperative sentences are not allowed, since the subject must be written in the sentence explicitly using nouns, pronouns, or noun phrases.

The Talk2Prolog consists of five Python files:

- `main.py`, which contains 124 LOC.

- `preprocessor.py`, a module which provides 3 functions and contains 180 LOC.
- `coref_handler.py`, a class which provides 2 methods and contains 39 LOC.
- `semantic_parser.py`, a class which provides 9 methods and contains 342 LOC.
- `prolog_handler.py`, a class which provides 4 methods and contains 72 LOC.

The main file orchestrates the different operations needed to represent the original sentence. In particular, it is divided into four parts:

1. Common parameters
2. Pre-processing
3. Semantic parsing
4. Prolog handling

6.1 Common parameters

The main library used for this project is spaCy and the default loaded model is the “`en_core_web_lg`” model, which is used in the pre-processing and semantic parsing phases.

There are also configuration flags that give the user some flexibility to modify the approach used to translate the input sentence into a Prolog expression. The default approach is the one described by the methodology created by us.

- `chunk_arg`: If it is set to “True” allows the user to insert entire noun phrases as atoms, including articles. In this case, the words are concatenated using the underscore. Otherwise, if it is set to “False”, only nouns will be considered, and other words present in the noun phrase may or may not appear based on the other configurations, not necessarily concatenated to the noun (e.g. “The sky is blue” would be translated `blue(sky)).` in the first case, and as `blue(the_sky).` in the second.). Default: “False”.
- `conjugate_present`: If it is set to “True”, always use the present tense of the verb, independently of the tense used in the original sentence. If it is set to “False” and the past tense is used in the sentence, the verb is conjugated using the past simple. Otherwise, if it is set to “False” and the past tense is not used in the sentence, the verb is left unchanged with reference to the sentence. Default: “True”.

- `conjugate_third_person`: When the flag “`conjugate_present`” is set to “True”, if the flag “`conjugate_third_person`” is set to “True” as well, the verb in the present tense will be agreed to the subject. If it is “False”, the base form will be used for the present, regardless of the subject (e.g. “Does Alice eat cookies?” would become `?- eats(alice, cookies).` in the first case, and `?- eat(alice, cookies).` in the second.).
- `include_adv`: If it is set to “True” allows the user to include an adverb in the resulting expression. Otherwise, if it is set to “False”, adverbs are ignored in the resulting Prolog term. Default: “True”.
- `include_preposition`: If it is set to “True” it allows the user to include a verb preposition in the resulting expression, concatenated to the verb using an underscore. Otherwise, if it is set to “False”, the preposition is ignored. Default “True”.
- `verb_as_relation`: If it is set to “True”, use the verb, independently if lexical or copular, to represent the relation between arguments (except in the case of the verb “to be”). If it is set to “False” only the complement and the subject will be used to form the resulting term if the verb is copular (e.g. “Alice seems sad.” is translated as `seems(alice, sad).` in the first case, and as `sad(alice).` in the second, since “seems” is copular.). Default: “True”.
- `char_as_var`: If it is set to “True” consider single uppercase letters as variables, except for the letter “I”. Otherwise, if it is set to “False”, uppercase letters are considered as atoms (e.g. “X is happy if X reads a book.”, “X” would be translated as a variable `X` in the first case, and as an atom `x` in the second). Default: “True”.
- `adj_as_term` = If it is set to “True” use adjectives as functors to create a term. Otherwise, concatenate adjectives to the front of the corresponding argument using an underscore (e.g. “Alice is a happy student.” would be translated as `happy(student(alice)).` in the first case, and as `happy_student(alice).` in the second.). Default: “True”.

There are also some common pre-processing flags called “`word2num`” (whose default is “True”, meaning that words representing numbers are turned into a numerical representation) and “`solve_coref`” (whose default is “manual”, meaning that the Talk2Prolog will use a custom coreference resolution function made by us using spaCy). These flags will be explained more in details in the dedicated section.

6.2 Pre-processing

Since a correct English sentence is expected as input, there is an initial pre-processing phase that ensures the first letter is capitalized and that the sentence ends with a full stop or a question mark.

After capitalization and punctuation of the input sentence, there is also a normalization process with the following steps:

1. Contractions are expanded and slang is fixed using the package contractions.
2. Periods that are not a full stop mark are eliminated.
3. The values expressed as words are converted to be expressed using digits using the package numerizer. This step is optional and can be turned off by setting the flag variable “word2num” to “False”, but the default is “True”. Indefinite articles “a” and “an” are not considered representations for the number one.
4. A coreference resolution step, which is used to replace all mentions of an entity with its best representative. It is possible to use a manual rule-based approach implemented with spaCy or the legacy deterministic coreference resolution implemented by Stanford CoreNLP. This step is optional and can be turned off by setting the flag variable “solve_coref” to “None”.

The rewritten sentence allows for a better recognition of the sentence elements by spaCy, and a consistent information representation in Prolog.

Finally, the sentence is divided into clauses. These clauses are then rewritten in a way to guarantee they are all independent, i.e. they express a complete thought, and then are individually parsed in the next phase.

All the functions necessary for this part are provided by the “preprocessor.py” module.

In the following subsections, we will provide more details on the coreference resolution and division of the sentence into clauses.

6.2.1 Coreference Resolution

6.2.1.1 Stanford

It is a legacy model that can be accessed by using a server provided by the Stanford NLP library. It can be enabled by setting the “solve_coref” flag to “stanford”.

6.2.1.2 Manual

To avoid the overhead due to the Stanford server setup and make the tool more independent, we decided to implement a simple coreference resolution system based only on spaCy which substitutes anaphoric pronouns with a noun already introduced in the phrase.

The idea was to use the fine part-of-speech tagging of the tokens to get the following nouns and pronouns, based on the order of appearance in the sentence:

- Singular nouns
- Plural nouns
- Personal subject, object, and reflexive pronouns.

Singular personal pronouns, whether subject or object, are substituted with the first occurrence of a singular noun in the phrase. The same goes for plural personal pronouns.

Pronouns that appear at the start of the sentence are left unchanged, since they do not refer to any noun mentioned previously. The substitution also does not happen if the singular pronoun is “it” and the noun has as the named entity “PERSON”.

The substitution does not happen if the pronoun appears at the start of the sentence, since it does not refer to any noun mentioned previously, and if the pronoun is “it” and the noun has as named entity type “PERSON”.

Of course, there is a great margin for improvement, such as taking into consideration the position of the noun and corresponding pronouns.

Another improvement is pairing singular nouns and pronouns based on gender, using a package such as Gender Guesser, although eventual dependencies and ethical concerns should be taken into account in these cases.

Distinguishing between the use of the plural or singular “they” pronoun is also a possible improvement.

This custom coreference resolution can be enabled by setting the “solve_coref” flag to “manual”, which is the default value.

6.2.2 Sentence to clauses

The last part of the pre-processing phase consists of diving the sentence into the clauses that compose it. These clauses are further divided into more clauses if there is the presence of compound elements.

The goal is to obtain a list of clauses that represent the sentence with one subject, one verb, and an eventual object or complement, which can be passed to the semantic parser individually to compose the final term.

In the case of complex sentences, to respect Prolog order, the independent clause which contains the conclusion appears before in the list than the clauses derived from the dependent clause.

This means that a sentence like “If Alice and Bob hate the book, then Charlie is sad.” is rewritten as “Charlie is sad if Alice and Bob hate the book.”.

6.3 Semantic parsing

The semantic parsing consists of iterating through the clauses identified in the pre-processing phase and transforming them into a corresponding Prolog term by assigning to the elements identified in the sentence the role of “relation”, “argument 1”, or “argument 2”.

All terms are saved and merged to obtain the final Prolog expression. More terms are concatenated together based on the presence of conjunctions in the original sentence, such as “and”, “or” and “if”.

This phase has access to a text file containing a list of some built-in Prolog predicates. For the purpose of this thesis, we inserted around twenty built-in predicates, but the file can be further extended to include more predicates in an eventual evolution of the tool.

This file is used to check if a predicate will need an eventual prefix (concatenated with an underscore) to avoid conflict with the built-in predicates in Prolog, giving more flexibility to the user. The default prefix is “my”.

The semantic parsing phase is responsible for verb conjugation, which is particularly useful for handling queries, passive sentences and homogenization of the tense used. It is done by using the spaCy extension `pyinflect`.

6.4 Prolog handling

The Prolog handling is done by using the library `PySwip`, which provides a dependency-free Python-Prolog interface, which allows us to insert new clauses into the knowledge base Prolog file and query this file.

This library is called inside the class `PrologHandler`. When an object of this class is created inside the main, the knowledge base provided as input by the user is loaded, and the Prolog

interpreter is initialized.

The class exposes two methods:

- Query
- Insert

6.4.1 Query

The method “query” allows the Talk2Prolog to query the knowledge base loaded previously during runtime.

The method receives the goal as parameter and returns no output.

If the result of querying the goal is “True”, the method prints the result, which can be “True” or “False”. If there are variables, the method also prints the bindings as the solution.

If the result of querying the goal is “False”, we needed to distinguish whether it was due to an error in the goal or just the actual result of the query.

For this reason, we first try to assert (and retract) only the term. If it succeeds, we return “False”, because it means the goal was well-formed and the negative result from the querying was due to the failure of proving the goal.

Otherwise, if the assertion (and consequently the retraction) of the term is not successful, the user is informed through an error message.

6.4.2 Insert

The method “insert” allows the Talk2Prolog to insert a new Prolog clause in the knowledge base loaded previously during runtime.

The clause is also written into the actual file corresponding to the knowledge base. In this way, if the user loads the knowledge base again, it will be updated from the last run of the program.

Before writing the clause into the knowledge base file, the predicate indicators “dynamic” and “discontiguous” for the predicate of the clause are inserted, indicating the.

The dynamic predicate indicator informs the interpreter that the definition of the predicate may change during execution.

The discontinuous predicate indicator informs the system that the clauses of the specified predicate might not appear next to each other in the knowledge base file.

If the insertion operation is not successful, the user is informed through an error message.

Chapter 7

Experiments

In this section, we will cover the results of the experiments performed with Talk2Prolog and GPT 5.2, which is the state-of-the-art GPT model at the time this thesis was written.

The objective is to analyze the difference between using a rule-based, specific approach with minimal deep-learning and a general-purpose large language model (LLM), which is based on generative AI and transformers.

The experiments are divided between syntactic and semantic testing.

- The syntactic testing consists of verifying if Talk2Prolog and GPT are able to create Prolog expressions that are written correctly according to the language rules and checking how similar they are to the corresponding expressions of the dataset.
- The semantic testing consists of verifying if Talk2Prolog and GPT can create Prolog expressions that not only are syntactically correct, but also convey the intended meaning of the original sentence.

We also measured the translation time for both Talk2Prolog and GPT to compare their performances.

The experiments were conducted on a Lenovo Windows 10 laptop with the following processor characteristics: AMD Ryzen 7 3700U with Radeon Vega Mobile Gfx, 2300 Mhz, 4 core, 8 logic processors.

7.1 Talk2Prolog settings

We used the default settings for the flags for our test executions, as explained in Sec. 6.1.

7.2 GPT settings

We used Python to connect to OpenAI’s API and to interact with PySwip, inserting and querying the results returned by GPT.

For the model response creation, in addition to the user input and model, we passed the temperature and top_p parameters equal to 0, in order to obtain results as deterministic as possible, although it is impossible to guarantee 100% determinism.

For the instructions, we had four versions, which are the result of the combination of the following characteristics:

- Instruct GPT to receive all the sentences at once as input (offline) or only one sentence at a time, processing them independently. This approach is used to verify how consistent GPT is depending on how much context it has.
- Inform GPT on some aspects of our methodology, to obtain a result more similar to our tool and the dataset representations, or use GPT as-is. This approach is used to compare the Prolog programming practices acquired by GPT during its training with the ones we formalized.

The instructions for the *online uninformed version* are as follows:

- You are a coding assistant that converts a single natural-language sentence into one Prolog expression.
- Output exactly one Prolog fact, rule, or query (no explanations).

The instructions for the *online informed version* are as follows:

- You are a coding assistant that converts a single natural-language sentence into one Prolog expression.
- Output exactly one Prolog fact, rule, or query (no explanations).
- Nested compound terms are allowed.
- Do not insert dummy subjects ("it" or "there" in expletive constructions)
- Every term must have arity 0, 1, or 2.

- Consider subject-verb agreement.

The instructions for the *offline uninformed version* are as follows:

- You are a coding assistant that converts natural-language sentences into Prolog expressions.
- For each sentence (which are separated by a new line), output exactly one Prolog fact, rule, or query (no explanations).

The instructions for the *offline informed version* are as follows:

- You are a coding assistant that converts natural-language sentences into Prolog expressions.
- For each sentence (which are separated by a new line), output exactly one Prolog fact, rule, or query (no explanations).
- Nested compound terms are allowed.
- Do not insert dummy subjects ("it" or "there" in expletive constructions)
- Every term must have arity 0, 1 or 2.
- Consider subject-verb agreement.

In the tables reporting the results for this section, we used the following labels to indicate the four GPT versions:

- ON-I GPT: online informed GPT.
- ON-U GPT: online uninformed GPT.
- OFF-I GPT: offline informed GPT.
- OFF-U GPT: offline uninformed GPT.

Syntax Errors	Talk2Prolog	ON-I GPT	ON-U GPT	OFF-I GPT	OFF-U GPT	Total expressions
Facts Websites	1	0	0	0	0	79
Facts Mix	4	1	2	0	0	257
Facts Variables	0	0	0	0	0	3
Queries Websites	5	0	0	0	0	34
Queries Mix	1	0	0	0	0	21
Rules Websites	4	0	0	0	0	17
Rules Mix	2	0	0	0	0	21
0-arity facts (GPT)	0	0	1	0	0	79
1-arity facts (GPT)	0	1	2	0	0	98
2-arity facts (GPT)	1	0	0	0	0	96
Running Examples	0	1	1	0	0	47
Total	18	3	6	0	0	752

Table 7.1: Number of sentences that were not translated or were translated into a syntactically incorrect Prolog expression. See Sec. 7.2 for GPT labels.

7.3 Syntactic testing

For the syntactic testing, we considered two aspects:

- Well-formed expressions: We counted how many sentences were not translated by the tool and how many were not correct Prolog expressions considering the language syntax rules.
- Logical equivalence: We compared syntactically correct expressions with their dataset counterparts to verify if they unify and if the terms are of the same type.
- String similarity: We compared how similar a generated Prolog expression was to its equivalent in the dataset considering string edit distance.

7.3.1 Well-formed expressions

For the syntactic testing, the first thing we considered were how many translations are correct Prolog expressions according to the language syntax rules.

In table 7.1, we report how many sentences raised errors during the translation process or when trying to insert or query the corresponding Prolog expression with incorrect syntax.

In table 7.2, we report the percentage of sentences that resulted in a syntactic error which with reference to the total number of sentences.

Considering the tests we have performed, Talk2Prolog was the only tool that encountered sentences it was not able to generate any translation due to pre-processing or semantic errors.

Syntax Errors	Talk2Prolog	ON-I GPT	ON-U GPT	OFF-I GPT	OFF-U GPT
Facts Websites	0,013	0,000	0,000	0,000	0,000
Facts Mix	0,016	0,004	0,008	0,000	0,000
Facts Variables	0,000	0,000	0,000	0,000	0,000
Queries Websites	0,147	0,000	0,000	0,000	0,000
Queries Mix	0,048	0,000	0,000	0,000	0,000
Rules Websites	0,235	0,000	0,000	0,000	0,000
Rules Mix	0,095	0,000	0,000	0,000	0,000
0-arity facts (GPT)	0,000	0,000	0,013	0,000	0,000
1-arity facts (GPT)	0,000	0,010	0,020	0,000	0,000
2-arity facts (GPT)	0,010	0,000	0,000	0,000	0,000
Running Examples	0,000	0,021	0,021	0,000	0,000

Table 7.2: Percentage of sentences that were not translated or were translated into a syntactically incorrect Prolog expression. See Sec. 7.2 for GPT labels.

Both Talk2Prolog and GPT might generate incorrect Prolog expressions, in particular, the online versions of GPT.

An example of an incorrect Prolog expression generated by Talk2Prolog is `quantity(years , 32(john))..`

Some examples of incorrect Prolog expressions generated by GPT online are `?- write(charlie, what).` and `fail..`. Many of the errors related to GPT are often related with the use of reserved keywords in Prolog.

Another common situation that returned an error for GPT was translating one sentence with more than one Prolog expression, despite the prompt instructing otherwise. For example, `circle(green). box(green).` was returned by offline uninformed GPT as a translation for “There is a green circle.”. Not only did it use two facts for the sentence, it also inserted the word “box”, probably because the previous two sentences were “There is a blue box.” and “There is a red box”.

7.3.2 Logical equivalence

We decided to create a Prolog script to compare two syntactically correct Prolog expressions considering:

- Unification
- Type of the arguments (in this case, being a variable or not).

L.E. Check	Talk2Prolog	ON-I GPT	ON-U GPT	OFF-I GPT	OFF-U GPT
Facts Websites	0,56962	0,69620	0,68354	0,68354	0,68354
Facts Mix	0,38911	0,53696	0,47860	0,52918	0,56809
Facts Variables	1	0	0	0	0
Queries Websites	0,47059	0,38235	0,64706	0,14706	0,14706
Queries Mix	0,28571	0,38095	0,42857	0,33333	0,33333
Rules Websites	0,29412	0,52941	0,64706	0,52941	0,52941
Rules Mix	0,33333	0,33333	0,38095	0,33333	0,38095
0-arity facts (GPT)	0,62025	0,69620	0,64557	0,64557	0,72152
1-arity facts (GPT)	0,92857	0,98980	0,86735	1	1
2-arity facts (GPT)	0,92708	0,95833	0,95833	0,95833	0,95833
Running Examples	1	0,61702	0,65957	0,59574	0,57447

Table 7.3: Logical equality ratio calculated by dividing how many checks were considered valid over the total number of expressions (including the ones that resulted in an error). See Sec. 7.2 for GPT labels. The highest ratio for each sheet is in bold.

If the terms of the expressions unify and have arguments in same number and of the same type, the check for logical equivalence is considered “VALID”. Otherwise, it is “INVALID” and the two expressions cannot be considered logically equivalent.

If the expression is a rule, we first check if the number of terms present considering the body and the head is the same of the corresponding expression of the dataset. If it is not, we consider the check “INVALID”. If it is, we proceed checking term by term, matching according to the position. If only one term does not correspond, the check is considered “INVALID”.

If the body presents a conjunction “;” or a disjunction “,”, if the first check is considered “invalid”, we try a second and last check, but this time inverting the order of one of the expressions, since both operators are symmetric. If again it fails, we consider the check to be definitely “INVALID”.

For the sentences that were not generated or were not syntactically correct, we used the check label “ERROR”.

If we consider as an example the sentence “Who likes skiing?” is translated as `?- likes(X, skiing).` in the dataset. Both GPT online versions translate in this way, while Talk2Prolog translate the sentence as `?- likes(Who, skiing)..`

These two translations are logically equivalent, therefore both are considered valid: the only difference is the naming convention for the variables, but the grammar rules of Prolog are still respected in both cases, the position of the variable and of the atom as well, and the predicates unify with the version of the dataset.

The results obtained are reported in Tab. 7.3.

From the results, it is possible to notice that Talk2Prolog usually performs slightly worse than GPT in terms of logical equality, although it is not always the case (e.g. in “Facts Variables”). A possible reason for this is not using the same functors as the dataset, what will be discussed more in details in Sec. 7.3.3. Regarding GPT, the uninformed versions perform better than the informed ones.

7.3.3 String similitary

To compare the results obtained with Talk2Prolog and all GPT versions to the Prolog representations in the dataset considering string similarity, we decided to use the “ratio” function from the Python library Levenshtein (which is *case insensitive*).

The “ratio” function returns the *indel similarity* between two strings, which is calculated in the following way:

$$1 - \frac{d}{(l_1 + l_2)} \quad (7.1)$$

Where:

- d is the distance between the two strings, calculated by taking the minimum number of insertions and deletions required to change one sequence to the other.
- l_1 and l_2 are the lengths of the first and second strings, respectively.

For the sentences for which Talk2Prolog returned an error before being able to generate a translation, we considered the similarity to be 0.

For the sentences that were translated into Prolog expressions instead, but are syntactically incorrect, we decided to still calculate the Levenshtein similarity ratio just as we did for the correct expressions, since we wanted to use it as a measure indicating how much it differs from a correct Prolog expression for the same sentence.

There are a few disclaimers to be made about the measures:

- There is no absolute standard to follow when it comes to representing information in Prolog. Therefore, a string not matching what is reported in the dataset is not a synonym of an incorrect representation. For example, some people might prefer to use the predicate “dislikes” instead of “not_likes”, but both predicates often convey the same meaning in natural language.

Levenshtein (%)	Talk2Prolog	ON-I GPT	ON-U GPT	OFF-I GPT	OFF-U GPT
Facts Websites	87,59	91,92	91,10	91,55	91,81
Facts Mix	80,51	89,94	88,32	89,97	92,30
Facts Variables	71,52	71,52	70,81	71,52	68,12
Queries Websites	71,10	90,40	90,50	88,77	82,26
Queries Mix	69,04	80,57	78,32	75,62	70,94
Rules Websites	68,92	92,61	94,27	93,51	92,02
Rules Mix	82,75	83,08	87,86	85,96	89,08
0-arity facts (GPT)	87,78	89,67	88,59	88,86	89,91
1-arity facts (GPT)	98,34	99,47	94,36	100,00	100,00
2-arity facts (GPT)	98,07	99,72	99,72	99,72	99,72
Running Examples	100,00	95,52	92,19	91,66	90,25

Table 7.4: Average Levenshtein ratio score. See Sec. 7.2 for GPT labels. The highest score for each sheet is in bold.

- The data generated by GPT and Talk2Prolog are more biased in favor of the respective tools used. Therefore, the sheets labeled as “GPT” and “Mix” are more likely to favor GPT, since it was involved in the process of their creation, and the sheet “Running Examples” was made taking into consideration what Talk2Prolog could handle. The only sheets that could be considered unbiased are the ones labeled as “Websites”.
- The Levenshtein ratio can be low for translations that are logically equivalent (e.g., `?- likes(Someone, Someoneelse).` is equivalent `?- likes(X, Y).`) and high for translations that are *not* logically equivalent (e.g., `?- likes(John, Mary).` is not equivalent to `?- likes(john, mary).`).

For this reason, Levenshtein is not as relevant for our purpose as the logical check, but we wanted to include it anyways to quantify how closely two Prolog expressions resemble each other in their written form, even if they do not share the same meaning.

In Tab. 7.4, the results obtained for the Levenshtein ratio are reported.

One interesting example from the dataset is the sheet “Facts Variables”. Talk2Prolog in this case passed all logical equality checks when compared to the database, while all versions of GPT failed all logical equality checks. Yet, their Levenshtein scores in 7.4 are similar.

This is a consequence of the Levenshtein function used being case-insensitive, while Prolog is not. Therefore, it assigns the same score to these expressions:

- Sentence “Everyone is a parent!”
 - Dataset: `parent(X).`

- Talk2Prolog: `parent(Everyone).`
- GPT online informed: `parent(everyone).`
- Levenshtein ratio: 66,77%
- Sentence “Everyone likes Susie.”
 - Dataset: `likes(X, susie).`
 - Talk2Prolog: `likes(Everyone, susie).`
 - GPT online informed: `likes(everyone, susie).`
 - Levenshtein ratio: 75,68%
- Sentence “John likes everybody.”
 - Dataset: `likes(john, X).`
 - Talk2Prolog: `likes(john, Everybody).`
 - GPT online informed: `likes(john, everybody).`
 - Levenshtein ratio: 75,68%

It is possible to notice that not only there is a penalization due to the use of the words “everyone” and “everybody” as variables instead of using “X”, but also using these words as variables or atoms is considered the same by the Levenshtein function.

Observing the measures, it is possible to notice that there is usually at least one GPT model that performs better than Talk2Prolog for most sheets in terms of string similarity.

The most significant difference was around 25% considering the online uninformed version of GPT for the sheet “Rules Websites”. But in many cases the difference between Talk2Prolog and the GPT versions is less than 10%.

A common situation in which Talk2Prolog struggles is with words that can act as both verbs and nouns, depending on the context, such as “studies” or “can”, usually resulting in unnatural translations, such as `csc135(charlie_studies).` or `cans(jaya, cook).`, for the sentences “Charlie studies csc135.” and “Jaya can cook.”, respectively. This depends on the result returned by the part-of-speech and dependency tagging of spaCy. In fact, some experiments with a more accurate model, “trf”, instead of “lg”, showed better results

Correct answer	Talk2Prolog	ON-I GPT	ON-U GPT	OFF-I GPT	OFF-U GPT
Accuracy	0,88372	0,32558	0,53488	0,74419	0,95349
Total correct answers	38	14	23	32	41
Total incorrect answers	5	29	20	11	2

Table 7.5: Accuracy and number of correct and incorrect answers for each translation tool over 43 semantic testing examples. See Sec. 7.2 for GPT labels.

in this case, but these are outside of the scope of this thesis due to the use of transformers by “trf”.

However, there are also a few situations in which Talk2Prolog actually performed better than one of the GPT versions, such as in the case of “1-arity facts (GPT)”, in which Talk2Prolog got around 4% higher result when compared to GPT online uninformed.

Regarding only the GPT versions, ultimately no GPT version consistently achieves a higher Levenshtein score when compared to the others versions.

There are cases in which more specific instructions work to obtain more accurate results, such as in the case of “Queries Websites”, and others in which more general instructions suit the task better, such as in the case “1-arity facts (GPT)”.

The choice most likely depends on the complexity of the sentence to translate.

7.4 Semantic testing

For the semantic testing, by querying our knowledge base containing rule and facts, we wanted to verify mainly three aspects:

- The capacity of the program to answer correctly to simple, straightforward yes/no questions and wh- questions.
- The capacity of the program to translate sentences with the same meaning in the active and passive voice in the same way.
- The capacity of the program to perform deductive reasoning by keeping a coherent naming system for predicates and arguments.

We used inference rules as inspiration for the test cases, but not in a strict way. We also considered other factors as well.

In Tab. 7.5, we reported the total number of correct and incorrect answers to the series of tests we submitted to all five tools.

This is the ranking of the tools performance based on accuracy:

1. Offline uninformed GPT
2. Talk2Prolog
3. Offline informed GPT
4. Online uninformed GPT
5. Online informed GPT

Talk2Prolog, in this case, was able to answer more correctly to the queries, losing in terms of accuracy only to offline uninformed GPT.

7.4.1 Yes/no questions

We might want to ask our knowledge base whether a fact representing an information is true or not.

For example, if we consider the sentence “Bob likes tea.” true, and then ask the question “Does Bob like tea?”, we expect a positive answer.

All five tools did the following translations:

- The sentence “Bob likes tea.” was represented as `likes(bob, tea)..`
- The sentence “Does Bob like tea?” was represented as the query `?- likes(bob, tea)..`, which returned the result “true”.

For another example, if we consider both the sentences “Alice plays the piano.” and “Alice plays the violin.”, if we ask “Does Alice play anything?”, we again expect a positive answer (because Alice plays both the piano and the violin).

All tools did the following translations:

- The sentence “Alice plays the piano.” was represented as `plays(alice, piano)..`
- The sentence “Alice plays the violin.” was represented as `plays(alice, violin)..`

The only difference was for the sentence “Does Alice play anything?”, which was translated as `?- plays(alice, _).` for all GPT versions and `?- plays(alice, Anything).` by Talk2Prolog.

Both Talk2Prolog and GPT again returned “true” (Talk2Prolog also specified the answer due to the use of the variable instead of the placeholder) and were considered correct.

7.4.2 Question words

We could also ask open ended questions to our knowledge, not only expecting a static answer.

For example, if we consider the following sentences:

1. Bob likes playing board games.
2. Bob likes eating fruits.
3. Bob likes football.
4. What does Bob like?

We expect as an answer to the last question that Bob likes playing board games, eating fruits and football.

Talk2Prolog and GPT online informed translated them in the following way:

1. `likes(bob, playing(board_games)).`
2. `likes(bob, eating(fruits)).`
3. `likes(bob, football).`
4. `?- likes(bob, What).`

GPT offline informed instead used the variable “X” instead of “What” for the last translation (`?- likes(bob,X).`)

The last query returned the three expected answers for Talk2Prolog and GPT informed, which is the correct result.

GPT online uninformed instead translated the sentences in the following way:

1. `likes(bob,playing_board_games).`
2. `likes(bob, eating(fruits)).`
3. `likes(bob, football).`
4. `?- likes(bob, X).`

GPT offline uninformed instead translated the sentences in the following way:

1. `likes(bob,playing_board_games).`
2. `likes(bob,eating_fruits).`
3. `likes(bob, football).`
4. `?- likes(bob, X).`

The last query of both GPT uninformed versions returned the three expected answers for Talk2Prolog and GPT informed, which is the correct result.

We can see some style patterns, like using nesting instead of the underscore and vice-versa or the naming of the variables, but all tools keep semantic consistency in this example.

7.4.3 Passive vs active voice

The examples using modus ponens were made to verify the capacity of the tools to queries with the same semantic meaning but different voices in the same way.

For example, if we consider the following sentences:

1. Active voice: Bob writes the articles.
2. Passive voice: The article is written by Bob.

Although the voice used is different, in both cases the action is to write, the agent is Bob and the recipient is the article.

Therefore, if we ask the question “Who writes the article?”, for example, we expected “Bob” to be answer.

All tools translated the active sentence as `writes(bob, article).`, while the second had different translations:

- Talk2Prolog and GPT offline uninformed translated the passive sentence as `writes(bob, article).`, using the agent as first argument, not the subject.
- The other versions of GPT instead translated the passive sentence as `written_by(article, bob).`, therefore keeping the subject as the first argument, not the agent.

Finally, the question translated by all tools as `?- writes(Who, article).`, which in the case of Talk2Prolog and GPT offline uninformed, gives as result “Bob” for both the translations of the active and passive sentences, which is semantically correct. Instead, for the other GPT versions, only the active sentence generates a fact that would unify with this query, which is semantically incorrect.

7.4.4 Modus Ponens

The examples using modus ponens were made to verify the capacity of the tools to perform simple deductive reasoning.

For example, we adapted a classical syllogism example in the following way:

1. If X is a human, then X is mortal.
2. Socrates is a human.
3. Is Socrates mortal?

These sentences were translated by all tools in the following way:

1. `mortal(X):- human(X).`
2. `human(socrates).`
3. `?- mortal(socrates).`

The last query returned “true” in all cases, which is the correct result.

7.4.5 Modus Tollens

The examples using modus tollens were made to verify the capacity of the tools to handle the negation as failure used in Prolog due to the closed-world assumption.

For example, if we consider the sentences:

1. It is sunny if the sky is blue.
2. Is it sunny?

If we do not state that “the sky is blue”, due to the closed-world assumption, we assume that asking “Is it sunny?” would result in a negative answer.

These sentences were translated by all tools in the following way:

1. `sunny :- blue(sky).`
2. `?- sunny.`

The last query returned “false” in all cases, which is the correct result.

7.4.6 Hypothetical Syllogism

The examples using hypothetical syllogism were made to verify the capacity of the tools to respond to queries that require recursion.

For example, if we consider the following sentences:

1. If there is snow, then the park is closed.
2. If the park is closed, then the children play at home.
3. There is snow.
4. Do the children play at home?

By stating the sentences in the exact same order they were listed above, we expect that the answer to the final question is positive, since there is snow, therefore the park is close, therefore the children play at home.

Talk2Prolog translated these sentences in the following way:

1. `closed(park):- snow.`
2. `play_at(children, home):- closed(park).`
3. `snow.`

4. `?- play_at(children, home).`

The last query returned “true”, which is the correct answer.

Both GPT offline versions instead translated the sentences in the following way:

1. `closed(park):- snow.`
2. `play_at_home(children):- closed(park).`
3. `snow.`
4. `?- play_at_home(children).`

The last query returned “true”, which is the correct answer.

Both GPT offline versions used the object “home” as part of the predicate, using it as an 1-arity fact instead of 2. It is a different translation style from Talk2Prolog, but it is still a consistent Prolog representation, since the answer returned by running the query is still the same and the predicates are coherent with each other.

Both GPT online versions instead translated the sentences in the following way:

1. `closed(park):- snow.`
2. `play_at_home(children):- closed(park).`
3. `snow.`
4. `?- play(children, home).`

The last query returned “false”, which is not the correct answer.

Both GPT online versions were inconsistent with the Prolog predicate used. At first, they included the object “home” in the predicate name, then they considered the object “home” as a second argument of the fact, using the predicate “play”.

This inconsistency lead to the wrong answer.

7.4.7 Conjunction

The examples using disjunction were made to verify the capacity of the tools to respond to queries that use rules in which the “and” operator is present in the body.

For example, we will consider the following sentences:

1. Mr. Smith is satisfied if Alice and Bob enjoy reading.
2. Alice enjoys reading.
3. Bob enjoys reading.
4. Is Mr. Smith satisfied?

By stating the sentences in the exact same order they were listed above, we expect that the answer to the final question is positive, since it is true that both Alice and Bob enjoy reading.

Talk2Prolog and GPT offline informed translated these sentences in the following way:

1. `satisfied(mr_smith):- enjoys(alice, reading), enjoys(bob, reading).`
2. `enjoys(alice, reading).`
3. `enjoys(bob, reading).`
4. `?- satisfied(mr_smith).`

The last query returned “true”, which is the correct answer.

GPT offline uninformed instead translated these sentences in the following way:

1. `satisfied(mr_smith):- enjoys_reading(alice), enjoys_reading(bob).`
2. `enjoys_reading(alice).`
3. `enjoys_reading(bob).`
4. `?- satisfied(mr_smith).`

The last query returned “true”, which is the correct answer as well.

It adopted a different style by including the object “reading” as part of the predicate, but kept consistency for the predicate during the translation of all sentences.

GPT online informed instead translated these sentences in the following way:

1. `satisfied(mr_smith):- enjoy(alice, reading), enjoy(bob, reading).`
2. `enjoys(alice, reading).`
3. `enjoys(bob, reading).`
4. `?- satisfied(mr_smith).`

The last query returned “false”, which is not the correct answer.

Again, the problem derives from the lack of consistency: in this case, not using subject-verb agreement in all predicates.

GPT online uninformed instead translated these sentences in the following way:

1. `satisfied(mr_smith):- enjoy_reading(alice), enjoy_reading(bob).`
2. `enjoys(alice, reading).`
3. `enjoys(bob, reading).`
4. `?- satisfied(mr_smith).`

The last query returned “false”, which is again not the correct answer.

In this case, both subject-verb agreement and the placement of the object were inconsistent and lead to the incorrect answer.

7.4.8 Disjunction

The examples using disjunction were made to verify the capacity of the tools to respond to queries that use rules in which the “or” operator is present in the body.

For example, if we consider the following sentences:

1. Bob is happy if Charlie moves to London or Paris.
2. Charlie moves to London.
3. Is Bob happy?

By stating the sentences in the exact same order they were listed above, we expect that the answer to the final question is positive, since it is true that Charlie moved either to London or Paris, since he moved to London.

Talk2Prolog translated the sentences in the following way:

1. `happy(bob):- moves_to(charlie, london); moves(charlie, paris).`
2. `moves_to(charlie, london).`
3. `?- happy(bob).`

GPT offline uninformed instead translated the sentences in the following way:

1. `happy(bob):- moves_to(charlie, london).`
2. `moves_to(charlie, london).`
3. `?- happy(bob).`

In both cases, the last query returned “true”, which is the correct answer, but it is worth noticing that it just worked because the second statement said that Charlie moves to London.

If instead we used “Charlie moves to Paris” for the second statement, both would have failed, since Talk2Prolog is not able to distribute the preposition “to” of the verb “move” across the predicates with compound objects and GPT offline uninformed generated only the rule for the case in which Charlie moves to London.

Both GPT online versions translated the sentences in the following way:

1. `happy(bob):- moves_to(charlie,london); moves_to(charlie,paris).`
2. `moves_to(charlie, london).`
3. `?- happy(bob).`

The last query returned “true”, which is the correct answer.

And the GPT offline informed version translated the sentences in the following way:

1. `happy(bob):- moves(charlie,london). happy(bob):- moves(charlie,paris).`

2. `moves(charlie, london).`

3. `?- happy(bob).`

The last query returned “true”, which is the correct answer.

This version used two different rules to express the disjunction, which is an equivalent alternative to use just one rule with the “;” operator.

7.5 Performance

We used the “perf_counter” function from the Python time module. This function returns the value of a performance counter (i.e., a clock with the highest available resolution) and is used to calculate the difference between two reference points.

We took the measurements in seconds, up to five decimal digits, and used the comma as a decimal sign.

To consider only the translation time, we did not include in the time counting the loading of the spaCy library for Talk2Prolog and the creation of the GPT client. For the same reason, for both Talk2Prolog and GPT, the interaction with PySwip were not included in the time counting too.

We performed the tests using the Python subprocess module, which passes the sentences as input for the Talk2Prolog implementation and all four GPT variations.

Talk2Prolog and the two GPT online variations receive one sentence as input per process instance, and both the resulting translation and the time measure of the generation are saved for each sentence.

In the case of the two GPT offline variations, the script is executed only once, receiving as input the entire file containing the sentences. The result containing all the translations and the total time spent in their generation is saved.

To obtain the average execution time metrics reported in Tab. 7.6, we did the following calculations:

- For Talk2Prolog and both online versions of GPT, we calculated the average of the column “Execution time” for each sheet.
- For both offline versions of GPT, we divided the total execution time spent to get the answer containing all translations and divided it by the total number of expressions obtained.

Exec time (s)	Talk2Prolog	ON-I GPT	ON-U GPT	OFF-I GPT	OFF-U GPT
Facts Websites	0,12039	1,91468	1,97670	0,09577	0,09127
Facts Mix	0,07830	2,04903	2,03756	0,10604	0,09118
Facts Variables	0,21319	2,67460	2,01144	0,76641	0,88424
Queries Websites	0,12384	2,50902	2,90415	0,14454	0,14106
Queries Mix	0,13843	2,04823	2,01211	0,18371	0,23133
Rules Websites	0,10068	1,94149	2,13315	0,22570	0,25027
Rules Mix	0,08138	2,03459	2,14515	0,23360	0,24378
0-arity facts (GPT)	0,04423	1,89924	1,79257	0,07109	0,05863
1-arity facts (GPT)	0,10921	1,98870	2,26991	0,06432	0,08539
2-arity facts (GPT)	0,06934	2,22122	2,04834	0,09706	0,08396
Running Examples	0,08491	1,92420	2,05331	0,11815	0,13481
Semantic Examples	0,05464	1,86613	1,92926	0,08980	0,08218

Table 7.6: Average execution time in seconds for Prolog expression translation. See Sec. 7.2 for GPT labels.

To obtain the total execution time reported in Tab. 7.7, we did the following calculations:

- For Talk2Prolog and both online versions of GPT, we summed all execution times of the individual translations for each sheet.
- For both offline versions of GPT, we reported the execution time measure for the translation of all sentences at once.

Talk2Prolog takes around one-tenth of a second to translate a sentence into its Prolog representation, sometimes even less time, while GPT online takes almost two seconds to do the same.

The offline version is significantly faster than the online version, taking around one-tenth of a second per sentence translation as well.

7.6 Results spreadsheet structure

The results obtained by our experiments were saved into another spreadsheet¹.

The results spreadsheet contains the same sheets mentioned in the dataset. Each row refers to the row of the same index in the corresponding sheet of the dataset.

For each tool, we used a block containing the following four columns to report the results:

¹Result spreadsheet, accessed on March 10, 2026

Exec time (s)	Talk2Prolog	ON-I GPT	ON-U GPT	OFF-I GPT	OFF-U GPT
Facts Websites	9,51058	151,25978	156,15911	7,56584	7,21002
Facts Mix	20,12412	526,60193	523,65358	27,25175	23,43226
Facts Variables	0,63958	8,02380	6,03431	2,29922	2,65273
Queries Websites	4,21060	85,30669	98,74098	4,91448	4,79611
Queries Mix	2,90709	43,01274	42,25428	3,85781	4,85802
Rules Websites	1,71149	33,00532	36,26348	3,83688	4,25458
Rules Mix	1,70889	42,72630	45,04816	4,90567	5,11936
0-arity facts (GPT)	3,49425	150,03998	141,61279	5,61577	4,63194
1-arity facts (GPT)	10,70271	194,89212	222,45113	6,30312	8,36818
2-arity facts (GPT)	6,65677	213,23726	196,64059	9,31734	8,06058
Running Examples	3,99071	90,43746	96,50534	5,55316	6,33615
Semantic Examples	6,06545	207,14093	214,14739	9,96815	9,12188

Table 7.7: Total execution time in seconds for Prolog expression translation for each tool and dataset sheet. See Sec. 7.2 for GPT labels.

1. The Prolog expression (the name of the column is the name of the tool used to generate it).
2. Generation execution time, expressed in seconds (s).
3. Levenshtein ratio expressed in percentage (%) for all sheets, except “Semantic Examples”. For the “Semantic Examples” sheet, there is instead the “Correct answer” column, which contains a boolean value that expresses whether a query result is correct or not.
4. “Check” which is a boolean value that indicates whether the Prolog expression in the results spreadsheet unifies and has the same term types as the corresponding dataset expression. For the “Semantic Examples” sheet, there is instead the “Result” column, which contains a the result returned if the expression resulted in a query.

This block of columns is repeated five times per sheet, one for each tool/configuration. They are ordered as follows:

1. One for block Talk2Prolog
2. One block for online informed GPT
3. One block for online uninformed GPT
4. One block for offline informed GPT

5. One block for offline uninformed GPT

After these twenty columns, there are the metrics columns:

1. Talk2Prolog metrics
2. Online informed GPT metrics
3. Online uninformed GPT metrics
4. Offline informed GPT metrics
5. Offline uninformed GPT metrics

For all the sheets, except for “Semantic examples”, these addition metrics report the following information for each tool used in the experiments:

- Execution time: time spent in the translation of the sentence into a Prolog expression.
 - Average
 - Standard deviation
 - Maximum value
 - Minimum value
 - Total execution time
- Levenshtein: string similarity ratio
 - Average
 - Standard deviation
 - Maximum value
 - Minimum value
- Check: logical equivalence check

- Check rate (number of valid checks divided by the total number of entries)
- Number of valid checks
- Number of invalid checks
- Number of errors
- Error rate (number of errors divided by the total number of entries)
- Coverage: how much of the dataset expressions could be replicated by the tools
 - Coverage rate
 - Number of expressions that are equal to the expressions of the dataset
 - Number of expressions that are *not* equal to the expressions of the dataset

For the sheet “Semantic examples” instead, we have the following metrics:

- Execution time: time spent in the translation of the sentence into a Prolog expression.
 - Average
 - Standard deviation
 - Maximum value
 - Minimum value
 - Total execution time
- Correct answer: check on the answers returned by the queries
 - Accuracy
 - Number of correct answers (how many values in the column “Correct answer” are marked “True”)
 - Number of incorrect answers (how many values in the column “Correct answer” are marked “False”)

7.7 Comparison between Talk2Prolog and GPT 5.2

Considering all the tests described in this section, we reached the conclusion that both Talk2Prolog and GPT are able to perform natural language to Prolog translations, but each tool has its advantages and limitations.

Starting from the syntactic perspective, Talk2Prolog is in general a less flexible tool than GPT, accepting a more limited range of sentence inputs it is able to translate in a syntactically acceptable way.

Indeed, Talk2Prolog is more likely than GPT to generate a syntactically incorrect expression or to not generate any expression at all. GPT online committed some syntactic errors as well, but they are all related to the improper use of Prolog keywords.

Still, both Talk2Prolog and GPT were able to translate most of the sentences contained in the dataset. If we compare the results obtained by the tools with the expected Prolog expressions, both in terms of string similarity and logical equality, GPT tends to perform slightly better, but in this case the advantage is not as consistent anymore, since there are a few situations in which Talk2Prolog performs better instead.

From a semantic perspective, Talk2Prolog presents high accuracy, returning more correct answers than all GPT versions, except for the offline uninformed GPT version.

The advantage is due to translation coherence: in fact, correctly representing the meaning of the original sentences and keeping the consistency in the generated expressions is fundamental for semantic tasks.

Since Talk2Prolog applies the same rules to parse all sentences if executed with the same settings, it is usually coherent with itself, representing the same grammatical structures in the same way.

GPT offline is also better than GPT online in keeping coherence between the translations, possibly due to the fact that by receiving all sentences together, GPT offline can keep memory of how it translated a certain sentence and apply the same process for other similar sentences as well or keep track of how sentences are connected to each other.

The online version of GPT instead often use subject-verb agreement in some cases, while in others they use the base form. They also sometimes use certain words as parts of the term name or as an argument. This variation leads to less accurate results when trying to retrieve data.

In time performance testing, Talk2Prolog typically outperforms GPT in speed, a difference that is more pronounced when comparing total execution times. This gap is especially significant when the tool is measured against the online versions of GPT, which all average execution times are over one second.

As expected, GPT offline is faster than GPT online. Again this is an advantage from receiving all sentences at once, since a single call to the API service results introduces less overhead.

Therefore, we can assume that Talk2Prolog might be a valid alternative to GPT for tasks that do not have all sentences readily available, rely on time performance, and require semantic consistency.

Overall, we observed that there are some stylistic differences between the Prolog expressions generated by GPT and Talk2Prolog, for example:

- GPT sometimes gives preference for the underscore for indefinite pronouns.
- GPT often uses the symbol “X” for variables instead of the question word, as Talk2Prolog does.
- GPT sometimes gives preference for more specific predicates than Talk2Prolog.

Although these choices might affect the final result in terms of comparison with the dataset expressions, they do not necessarily affect negatively the syntax and semantics of a Prolog program.

Chapter 8

Conclusions and Future Work

In conclusion, the contributions of this thesis consist of:

- The creation of a dataset containing Prolog expressions paired with an English sentence representing its meaning, both for syntax and semantics (856 pairs in total).
- The design of a methodology to translate from English to Prolog based on the shared practices adopted by Prolog users, where 15 examples of translation rules were validated by 33 participants to our survey.
- The implementation of the software Talk2Prolog based on the methodology (757 LOC in total)
- A comparison between the performance of Talk2Prolog and different versions of GPT in terms of syntactic and semantic results (GPT with a higher accuracy for the syntactic testings, while Talk2Prolog outperformed most GPT versions for the semantic testings), and time performance (Talk2Prolog usually faster than GPT, especially the online version of GPT, in the order of a tenth of a second).

Talk2Prolog emerges as an alternative tool to LLMs and generative AI with the following advantages:

- It is a deterministic tool with a consistent way of translating the same grammatical entities.
- It is frugal and lightweight, being suitable for local running.

- It is explainable and debuggable, allowing its use in situations where privacy is fundamental.

Although the software showed solid results, especially if compared to GPT considering coherence in translation and time performance, there is still room for improvements.

To select some possible future work, we took into consideration the opinions and suggestions collected in the survey.

One of the most suggested features was the inclusion of arithmetic comparison operators in the translation, to allow the user to perform calculations using numerical values:

- The expression “greater than” being represented by the operator “>” in Prolog.
- The expression “less than” being represented by the operator “<” in Prolog.
- The expression “greater than or equal to” being represented by the operator “>=” in Prolog.
- The expression “less than or equal to” being represented by the operator “=<” in Prolog.
- The expression “equal to” being represented by the operator “:=” in Prolog.
- The expression “different from” being represented by the operator “\=” in Prolog.

This idea could be further extended to similar expressions, such as “more than” being translated in the same way as “greater than”.

Differencing nouns that should become variables from atoms, in particular by focusing on whether they are common nouns or not, is also a possible extension.

Many issues raised in the survey regarded the lack of context for the sentences. For this reason, a possible extension would be to manage texts containing more than one sentence and keep track of references.

Another way to improve the methodology is to distinguish between proper and common nouns. Common nouns can be used as term names, wrapping an id or a name. Common nouns can also be translated as variables, if preceded by an indefinite article.

In the implementation, additional flags could be added to take into account users’ preferences for the translation of common nouns. This is feasible since spaCy has distinguished tags for common and proper nouns, both singular and plural:

- “NN” for singular, common noun.
- “NNP” for singular, proper noun.
- “NNS” for plural, common noun.
- “NNPS” for plural, proper noun.

It is also possible to extend the work to include terms of arity greater than two. One way to do so is by including in the methodology more complex clause structures, such as:

- SVOO clauses: This clause pattern consists of a subject (S), a ditransitive verb (V), an indirect object (O), and a direct object (O).
- SVOC clauses: This clause pattern consists of a subject (S), a complex transitive verb, a direct object (O), and an object complement (C).
- SVOA clauses: This clause pattern consists of a subject (S), a complex transitive verb, a direct object (O), and an adverbial (A). In this case, the adverbial functions as an object complement for the complex transitive verb.

These structures are derived from the SVO clause structure used in the methodology. They could be represented in Prolog using 3-arity terms, where the term name corresponds to the verb, the first argument to the subject, and the second argument to the direct object, as seen for SVO clauses. The third argument instead would correspond to:

- The indirect object, for SVOO clauses.
- The object complement, for SVOC clauses.
- The adverbial, for SVOA clauses.

A possible extension could be adding words and collocations to express the degree of certainty of a sentence, such as “maybe”, “definitely”, and “likely”, associating to their Prolog representations a value representing the certainty of the representation. This addition could possibly be represented using ProbLog [RKT07], a probabilistic extension of Prolog where every clause is labeled with a probability.

A newer version of the rule-based coreference resolution approach would be ideal, instead of relying on the Stanford Library, which is out of date. The other version relies on transformers or neural approaches. Extending the coreference resolution system developed

ad hoc for this project using the suggestions mentioned in 6.2.1.2 could be a possible solution.

It would also be interesting to do the opposite task, translating from Prolog to natural language, to answer the queries.

Regarding the experiments, testing informed GPT with a significantly more detailed instruction based on the methodology could be an interesting future analysis.

Additionally, testing Talk2Prolog against other LLMs and other last generation tools could also provide more insight about our software capabilities. This will be the focus of an ongoing activity, a new master thesis that just started and will move on, building upon the results of this one.

Acknowledgements

Survey acknowledgements

We gratefully thank the participants to the dataset validation survey listed below in alphabetical order, and those that contributed anonymously.

Giorgio Roberto Acossi, Alessandro Acquilino, Davide Ancona, Andreas Brännström, Alessia Donata Camarda, Samuele Capani, Sara Caviglia, Veronica Dahl, Alessandro Dal Palu', Zahra Daoui, Pierangelo Dell'Acqua, Matteo Ferrari, Marco Gavanelli, Giovambattista Ianni, Robert Kowalski, Stefani Luca, Luca Ninivaggi, Stefano Passaggio, Enrico Pezzano, David Poole, Francesco Ricca, Giuseppe Rosolini, Sara Samiei, Iryna Savchuk, Clive Spenser, Gioacchino Sterlicchio, Veneta Tabakova-Komsalova, Federica Tamerisco, Felix Quirin Weitzkämper, and Michael Winikoff.

Personal acknowledgements

```
thanks(laura_fuciarelli, viviana_mascardi).  
thanks(laura_fuciarelli, andrea_gatti).  
thanks(laura_fuciarelli, angelo_ferrando).  
thanks(laura_fuciarelli, giorgio_delzanno).  
thanks(laura_fuciarelli, her_friends).  
thanks(laura_fuciarelli, her_family).
```

I would like to express my deepest gratitude to my supervisor, Professor Viviana Mascardi. She was extremely kind, patient, and competent, always available to provide me with

feedback and accommodate my requests. I could not have asked for a better mentor than her.

I would also like to express my deepest appreciation to Andrea Gatti and Angelo Ferrando for their expertise. Your suggestions were crucial to me many times.

I am also extremely grateful to my examiner, Professor Giorgio Delzanno, for his approachability and to my whole exam committee for their availability to assess my work.

Additionally, I am deeply indebted to everyone, anonymous or not, who dedicated some of their time to answering our survey. It was an important part of this thesis, and I value your feedback on my work.

I am also grateful to my classmates, coworkers, and friends for helping me when I needed it, while also providing me with some necessary breaks. In particular, I would like to mention Samuele Capani, who was my loyal work partner in many contexts and a great friend. I would also like to recognize Iryna Savchuk, who always had trust in my capabilities, sometimes more than I did.

Thanks should also go to Federico Ricciardi, Ion Menghini, and Luca Cavalca. You were the basis of my journey to becoming a computer scientist, and I know you cheered for me even when you were not around.

Lastly, I would like to thank my parents, who supported me during my entire journey. In particular, I would like to express my deepest gratitude to my mom. She endured everything with me, staying by my side even in the hard moments, forgiving me for every moment I was difficult to be around. When I truly thought I was never going to make it until the end, she gave me the strength and courage necessary to not give up on my dreams.

Chapter 9

Appendix: Selected Code Fragments

The manual coreference resolution, which is used by Talk2Prolog by default.

```
doc = nlp(output_sentence)
singular_nouns = [token for token in doc if token.tag_ in ["NN", "NNP"]]
plural_nouns = [token for token in doc if token.tag_ in ["NNS", "NNPS"]]
personal_pronouns = [token for token in doc if token.tag_ in ["PRP"] and
    token.i > 0]
replacement_dict = dict()
for pp in personal_pronouns:
    if pp.text.lower() in ["they", "them", "themselves"] and len(
        plural_nouns) > 0:
        replacement_dict[pp.text] = plural_nouns[0].text
    elif pp.text.lower() in ["he", "she", "it", "him", "her", "himself", "
        herself", "itself"] and len(singular_nouns) > 0:
        if not (singular_nouns[0].ent_type_ in ["PERSON"] and pp.text.lower
            () == "it"):
            replacement_dict[pp.text] = singular_nouns[0].text
for key, value in replacement_dict.items():
    output_sentence = re.sub(fr"\b{key}\b", value, output_sentence)
```

The function “sentence_to_clauses” that receives a sentence as input and returns the clauses that compose it. These clauses are rewritten in order to be all independent.

```
def sentence_to_clauses(input_text: str, nlp=None):
    def aux(doc):
        output_txt = []
        singular = True
        for token in doc:
            txt = token.text
            if token.dep_ == "nsubj":
                if token.tag_ in ["NNS", "NNPS"]:
                    singular = False
```

```

        elif token.tag_ in ["VBD", "VBP", "VBZ"]:
            if token.tag_ in ["VBP", "VBZ"]:
                if singular:
                    inflect = "VBZ"
                else:
                    inflect = "VBP"
            else:
                inflect = "VBD"
            txt = token._.inflect(inflect)
        elif token.dep_ != "nsubj" and token.tag_ in ["NN", "NNS"]:
            if singular:
                inflect = "NN"
            else:
                inflect = "NNS"
            txt = token._.inflect(inflect)
        output_txt.append(txt)
    return output_txt

doc = nlp(input_text)
cc = None
sconj = None
independent = []
after_then = False
left = []
right = []
first_sub_txt = ""
second_sub_txt = ""
for token in doc:
    if token.pos_ == "SCONJ":
        sconj = token
    elif token.text.lower() == "then":
        after_then = True
    elif not sconj or after_then:
        if token.dep_ != "punct":
            independent.append(token)
    else:
        if token.dep_ == "cc":
            cc = token
        elif token.dep_ != "punct":
            if not cc:
                left.append(token)
            else:
                right.append(token)

ind_clause = " ".join([i.text for i in independent])
right_verbs = [token for token in right if token.tag_.startswith("VB")]
left_verbs = [token for token in left if token.tag_.startswith("VB")]
right_subjs = [token for token in right if token.dep_ in ["expl", "nsubj"]

```

```

    "] or (token.dep_ == "conj" and (token.tag_.startswith("N") or token.
    tag_ == "PRP"))]
left_subjs = [token for token in left if token.dep_ in ["expl", "nsubj"]
    or (token.dep_ == "conj" and (token.tag_.startswith("N") or token.
    tag_ == "PRP"))]
dependent_clauses = None

if (len(left) > 0 and len(right) > 0):
    left_right_verbs = (len(left_verbs) > 0 and len(right_verbs) > 0)
    left_right_subjs = (len(left_subjs) > 0 and len(right_subjs) > 0)
    dependent_clauses = not (left_right_verbs and left_right_subjs)

if dependent_clauses:
    if len(right_subjs) > 0:
        if len(right_verbs) == 0:
            verb = left_verbs[0]
            start_index = left.index(verb)
            first_sub = left
            second_sub = left[0:start_index+1] + right
            first_sub_txt = [token.text for token in first_sub]
            second_sub_txt = [token.text for token in second_sub]
            first_sub_txt = " ".join(first_sub_txt)
            second_sub_txt = " ".join(second_sub_txt)
        else:
            verb = right_verbs[0]
            start_index = right.index(verb)
            first_sub = left + right[start_index:]
            second_sub = right
            first_sub_txt = aux(first_sub)
            second_sub_txt = aux(second_sub)
            first_sub_txt = " ".join(first_sub_txt)
            second_sub_txt = " ".join(second_sub_txt)
    else:
        left_txt = [token.text for token in left]
        right_txt = [token.text for token in right]
        verb = left_verbs[0]
        start_index = left.index(verb)
        offset = 1
        if right[0].tag_.startswith("VB"):
            offset = 0
        first_sub_txt = " ".join(left_txt)
        second_sub_txt = " ".join(left_txt[0:start_index+offset] +
            right_txt)
    else:
        left_txt = [token.text for token in left]
        first_sub_txt = " ".join(left_txt)
        right_txt = [token.text for token in right]

```

```

        second_sub_txt = " ".join(right_txt)

    clauses = [ind_clause]
    if sconj:
        clauses.append(sconj.text.lower())
    if first_sub_txt:
        clauses.append(first_sub_txt)
    if cc:
        clauses.append(cc.text.lower())
    if second_sub_txt:
        clauses.append(second_sub_txt)
    return clauses

```

The Prolog script used to check whether the generated solution by Talk2Prolog and GPT is logically equivalent to the one present in the dataset.

```

check(X, Y) :-
    ( var(X) ; var(Y)),
    !,
    X = Y.
check(X, Y) :-
    functor(X, F, N),
    functor(Y, F, N),
    X =.. [_|Xs],
    Y =.. [_|Ys],
    maplist(same_arg_type_and_unify, Xs, Ys).

same_arg_type_and_unify(A, B) :-
    (var(A), var(B) ; ground(A), ground(B)),
    !,
    A = B.

```

Bibliography

- [BPF⁺24] Flavio Bertini, Alessandro Dal Palù, Francesco Fabiano, Andrea Formisano, and Federica Zaglio. Concept2text: An explainable multilingual rewriting of concepts into natural language. In Emanuele De Angelis and Maurizio Proietti, editors, *Proceedings of the 39th Italian Conference on Computational Logic, Rome, Italy, June 26-28, 2024*, volume 3733 of *CEUR Workshop Proceedings*. CEUR-WS.org, 2024. URL: <https://ceur-ws.org/Vol-3733/paper14.pdf>.
- [CA24] Akshay Chaturvedi and Nicholas Asher. Learning semantic structure through first-order-logic translation. In Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen, editors, *Findings of the Association for Computational Linguistics: EMNLP 2024, Miami, Florida, USA, November 12-16, 2024*, volume EMNLP 2024 of *Findings of ACL*, pages 6669–6680. Association for Computational Linguistics, 2024. URL: <https://doi.org/10.18653/v1/2024.findings-emnlp.390>, doi:10.18653/V1/2024.FINDINGS-EMNLP.390.
- [CCM⁺24] Erica Coppolillo, Francesco Calimeri, Giuseppe Manco, Simona Perri, and Francesco Ricca. LLASP: fine-tuning large language models for answer set programming. In Pierre Marquis, Magdalena Ortiz, and Maurice Pagnucco, editors, *Proceedings of the 21st International Conference on Principles of Knowledge Representation and Reasoning, KR 2024, Hanoi, Vietnam. November 2-8, 2024*, 2024. URL: <https://doi.org/10.24963/kr.2024/78>, doi:10.24963/KR.2024/78.
- [CDM⁺24] Simone Caruso, Carmine Dodaro, Marco Maratea, Marco Mochi, and Francesco Riccio. CNL2ASP: converting controlled natural language sentences into ASP. *Theory Pract. Log. Program.*, 24(2):196–226, 2024. URL: <https://doi.org/10.1017/s1471068423000388>, doi:10.1017/S1471068423000388.

- [CDMT25] Simone Caruso, Carmine Dodaro, Marco Maratea, and Alice Tarzariol. A general framework for representing controlled natural language sentences and translation to KR formalisms. In *Proceedings of the Thirty-Fourth International Joint Conference on Artificial Intelligence, IJCAI 2025, Montreal, Canada, August 16-22, 2025*, pages 4401–4409. ijcai.org, 2025. URL: <https://doi.org/10.24963/ijcai.2025/490>, doi:10.24963/IJCAI.2025/490.
- [CGT89] Stefano Ceri, Georg Gottlob, and Letizia Tanca. What you always wanted to know about datalog (and never dared to ask). *IEEE Trans. Knowl. Data Eng.*, 1(1):146–166, 1989. doi:10.1109/69.43410.
- [Cov94] Michael A. Covington. GULP 3.1: An extension of Prolog for unification-based grammar. (IR), 1994. CASPR Research Reports. URL: <http://openscholar.uga.edu/record/25960>.
- [Den97] Daniel C Dennett. The role of language in intelligence. *Sprache und Denken/Language and Thought*, pages 42–55, 1997.
- [DWP12] David R Dowty, Robert Wall, and Stanley Peters. *Introduction to Montague semantics*. Springer Science & Business Media, 2012.
- [FKK08] Norbert E. Fuchs, Kaarel Kaljurand, and Tobias Kuhn. Attempto controlled english for knowledge representation. In Cristina Baroglio, Piero A. Bonatti, Jan Maluszynski, Massimo Marchiori, Axel Polleres, and Sebastian Schaffert, editors, *Reasoning Web, 4th International Summer School 2008, Venice, Italy, September 7-11, 2008, Tutorial Lectures*, volume 5224 of *Lecture Notes in Computer Science*, pages 104–124. Springer, 2008. doi:10.1007/978-3-540-85658-0_3.
- [FS95] Norbert E. Fuchs and Rolf Schwitter. Specifying logic programs in controlled natural language. *CoRR*, abs/cmp-lg/9507009, 1995. URL: <http://arxiv.org/abs/cmp-lg/9507009>, arXiv:cmp-lg/9507009.
- [Hud88] Rodney Huddleston. A comprehensive grammar of the english language, 1988.
- [KDC21] Robert Kowalski, Jacinto Dávila, and M Calejo. Logical english for legal applications. In *Conference: XAIF, Virtual Workshop on XAI in Finance*, 2021.
- [Kre88] Per Kreuger. A higher order logic parser for natural language implemented in lambda prolog. In Tim O’Shea and Vassil Stoyanov Sgurev, editors, *Artificial Intelligence III: Methodology, Systems, Applications - Proceedings of the Third International Conference on Artificial Intelligence: Methodology, Sys-*

tems, Applications, AIMS A 1988, Varna, Bulgaria, September 20-23, 1988, pages 299–306. North-Holland, 1988.

- [Kuh14] Tobias Kuhn. A survey and classification of controlled natural languages. *Comput. Linguistics*, 40(1):121–170, 2014. URL: https://doi.org/10.1162/COLI_a_00168, doi:10.1162/COLI_A_00168.
- [Lif19] Vladimir Lifschitz. *Answer Set Programming*. Springer, 2019. doi:10.1007/978-3-030-24658-7.
- [PW80] Fernando C. N. Pereira and David H. D. Warren. Definite clause grammars for language analysis - A survey of the formalism and a comparison with augmented transition networks. *Artif. Intell.*, 13(3):231–278, 1980. doi:10.1016/0004-3702(80)90003-X.
- [Qi09] Xiaochu Qi. An implementation of the language lambda prolog organized around higher-order pattern unification. *CoRR*, abs/0911.5203, 2009. URL: <http://arxiv.org/abs/0911.5203>, arXiv:0911.5203.
- [RG95] Anand S. Rao and Michael P. Georgeff. BDI agents: From theory to practice. In *Proceedings of the First International Conference on Multiagent Systems, June 12-14, 1995, San Francisco, California, USA*, pages 312–319. The MIT Press, 1995.
- [RKLY25] Hyun Ryu, Gyeongman Kim, Hyemin S. Lee, and Eunho Yang. Divide and translate: Compositional first-order logic translation and verification for complex logical reasoning. In *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025*. OpenReview.net, 2025. URL: <https://openreview.net/forum?id=09FiNmvmNMw>.
- [RKT07] Luc De Raedt, Angelika Kimmig, and Hannu Toivonen. Problog: A probabilistic prolog and its application in link discovery. In Manuela M. Veloso, editor, *IJCAI 2007, Proceedings of the 20th International Joint Conference on Artificial Intelligence, Hyderabad, India, January 6-12, 2007*, pages 2462–2467, 2007. URL: <http://ijcai.org/Proceedings/07/Papers/396.pdf>.
- [SKR24] Manuel A. Borroto Santana, Irfan Kareem, and Francesco Ricca. Towards automatic composition of ASP programs from natural language specifications. In *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence, IJCAI 2024, Jeju, South Korea, August 3-9, 2024*, pages 6198–6206. ijcai.org, 2024. URL: <https://www.ijcai.org/proceedings/2024/685>.

- [SSK⁺86] Marek J. Sergot, Fariba Sadri, Robert A. Kowalski, F. Kriwaczek, Peter Hammond, and H. T. Cory. The british nationality act as a logic program. *Commun. ACM*, 29(5):370–386, 1986. doi:10.1145/5689.5920.
- [TLC⁺21] Jidong Tian, Yitian Li, Wenqing Chen, Liqiang Xiao, Hao He, and Yaohui Jin. Diagnosing the first-order logical reasoning ability through logicnli. In Marie-Francine Moens, Xuanjing Huang, Lucia Specia, and Scott Wen-tau Yih, editors, *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing, EMNLP 2021, Virtual Event / Punta Cana, Dominican Republic, 7-11 November, 2021*, pages 3738–3747. Association for Computational Linguistics, 2021. URL: <https://doi.org/10.18653/v1/2021.emnlp-main.303>, doi:10.18653/V1/2021.EMNLP-MAIN.303.
- [Vas20] Yuli Vasiliev. *Natural language processing with Python and spaCy: A practical introduction*. No Starch Press, 2020.
- [YT23] Xiaocheng Yang and Yik-Cheung Tam. Exploring an LM to generate prolog predicates from mathematics questions. *CoRR*, abs/2309.03667, 2023. URL: <https://doi.org/10.48550/arXiv.2309.03667>, arXiv:2309.03667, doi:10.48550/ARXIV.2309.03667.
- [YXP⁺23] Yuan Yang, Siheng Xiong, Ali Payani, Ehsan Shareghi, and Faramarz Fekri. Harnessing the power of large language models for natural language to first-order logic translation. *CoRR*, abs/2305.15541, 2023. URL: <https://doi.org/10.48550/arXiv.2305.15541>, arXiv:2305.15541, doi:10.48550/ARXIV.2305.15541.
- [ZC12] Luke S. Zettlemoyer and Michael Collins. Learning to map sentences to logical form: Structured classification with probabilistic categorial grammars. *CoRR*, abs/1207.1420, 2012. URL: <http://arxiv.org/abs/1207.1420>, arXiv:1207.1420.